

(ρ)Metheus: Probabilistic Causal-Intelligence Oracle for Covenant-Bound Artificial Intelligence

Ling Xiao.¹ ✉

Metheus

Abstract

This technical document introduces the (ρ)Metheus Oracle, a protected Causal-Intelligence adjudication environment that mobilizes:

- audited artificial intelligence (AI) models,
- validated information pipelines,
- to compute *causal relations* from data.

Although (ρ)Metheus is a general purpose computational oracle, its initial deployment targets are the regulators, central/commercial banks, insurers, and other entities serving transnational trade in the Global South. Specifically, outputs of (ρ)Metheus will inform high-valued decisions pertaining to the disbursement of funds and claim resolutions. These are business-critical decisions that require both:

- regulator oversight *a priori* oracle deployment;
- and proper assignment of liabilities *a posteriori* oracle judgment.

This implies every step of the computation inside the (ρ)Metheus Oracle must be properly governed by state-level entities, so that the oracle satisfies the strong guarantee of:

- correct or Covenant-Compliant computations run to completion;
- while false or covenant-violating computations fail outright.

This requirement is addressed by statically typed programming languages, where programs that do not terminate fail at compile time. We enforce this design pattern in (ρ)Metheus. At the heart of the oracle is a *domain-specific programming language* called: Typed ρ -Calculus for probabilistic Causal-Intelligence. It is a two-tiered computational model or proof system whereby:

- *in the term universe* or the "analytic" level, (ρ)Metheus uses Pearl's do-Calculus to evaluate causal relations. In particular, the oracle uses information from validated data sources and summaries generated by audited large language models (LLMs) to compute causal proofs.
- *at the type universe* or the compliance level rests a meta-language extended by state regulators. The type-system governs the quality or legality of computation, so that illegal and false computations terminate immediately. Thereby providing a static guarantee that (ρ)Metheus-bound AI agents are well-behaved in accordance to legal constraints.

When verifying programs for contractual compliance, the (ρ)Metheus compiler uses a legalistic analogue of Floyd-Hoare logic, or Loare-Logic to generate proofs witnessing the Covenant-Compliance of Typed ρ -Calculus code written by analysts. This automated proof system shifts the burden of writing legally-compliant programs from analysts to the (ρ)Metheus compiler, thereby extending the operation domain of AI agents into more sensitive and regulated settings.

In other words, if code is law and AI agents are junior employees, then the (ρ)Metheus oracle is populated by a set of law-abiding workers. Meanwhile the Loare-Logic proof system is the enforcer of data and AI validity. The legal content of Loare-Logic is defined in the FairCovenant Foundation, a legislative body governing AI safety in trade along the Global South. Thus Loare-Logic constitutes to the *lingua-franca* of safeAI, and sits appropriately in the public domain as a public good. We brand this computational model distinguished by 1) validated data pipelines, 2) audited AI models used to discover causal relations, and 3) deep regulatory oversight governing the computation graph of the (ρ)Metheus oracle: **Covenant-Compliant Causal-Intelligence**.

¹ lingxiao@metheusai.xyz

Keywords and phrases Machine learning, statistics, artificial intelligence, causality, causal reinforcement learning, formal verifications, deep AI governance, AI safety, safeAI, programming language design, domain specific programming language, legalistic technology.

Contents

1	Introduction	4
1.1	Technical Stakeholder Perspective	4
1.1.1	Computational Asset Lifecycle in (pro)Metheus	4
1.1.2	An Example for Analysts	5
1.1.3	An Example for Regulators	6
1.1.4	An Example for Policy Holders	7
1.2	Broad Stakeholder Perspective	9
1.2.1	The (pro)Metheus Oracle Desiderata: a High Level Attempt	10
1.2.2	The (pro)Metheus Stakeholder Stack	11
1.3	The Greater Metheus Ecosystem	12
2	The Covenant-Type Correspondence	13
2.1	Preliminary Definitions and Examples	13
2.2	The Correspondence	14
3	Formal Specification of the (pro)Metheus Covenant	15
3.1	Introduction	15
3.2	Prior Art	15
3.3	Formal Specification of the (pro)Metheus Covenant	18
4	Formal Specification of Simply Typed ρ-Calculus	21
4.1	Technical Background	21
4.2	Typed ρ -Calculus Expressions	23
4.2.1	Elementary Typed ρ -Calculus Expressions	24
4.2.2	Group Composition over asset Expressions	24
4.2.3	Elementary Functions over asset Expressions	25
4.3	Loare-Logic: Axiomatic Semantics with Legalistic-Hoare Logic	26
4.3.1	Loare-Logic Introduction	26
4.3.2	Elementary Covenant-Judgement over Computational Assets	27
4.3.3	Semantics of data Composition w.r.t Covenant-Judgement	27
4.3.4	Semantics of agnt Composition w.r.t Covenant-Judgement	28
4.3.5	Semantics of Function Application w.r.t Covenant-Judgement	29
4.3.6	Proof of Covenant-Compliance with Loare-Logic	31
5	Extending (pro)Metheus with Causal-Intelligence	34
5.1	Why Estimating Causality is Valuable in Applied AI	34
5.1.1	A Brief History of AI	34
5.1.2	The Next Frontier of Applied AI is Causal-Intelligence	35
5.2	Technical Preliminary	37
5.2.1	The Forward and Backward Question of Causality	38
5.2.2	The Ladder of Causation	38
5.3	Case Study: Transnational Supply Chain Insurance	40

5.3.1	User Journey	40
5.3.2	The (ρro) Metheus Adjudication Game	41
5.3.3	Posing the Causal-Intelligence Adjudication Problem	42
5.4	Governing (ρro) Metheus Causal-Intelligence with Loare-Logic	45
6	Technical Background On Causal Reinforcement Learning	48
6.1	Causal Bayesian Networks and do-Calculus	48
6.1.1	Bayesian Network as the Underlying Data Structure	48
6.1.2	Operation on Data Structure with do-Calculus	49
6.2	Causal Reinforcement Learning	54
6.2.1	Reinforcement Learning and MDPs	55
6.2.2	Causal Reinforcement Learning with Known Causal Graph	56
6.2.3	Causal Reinforcement Learning with Unknown Causal Graph	60

1 Introduction

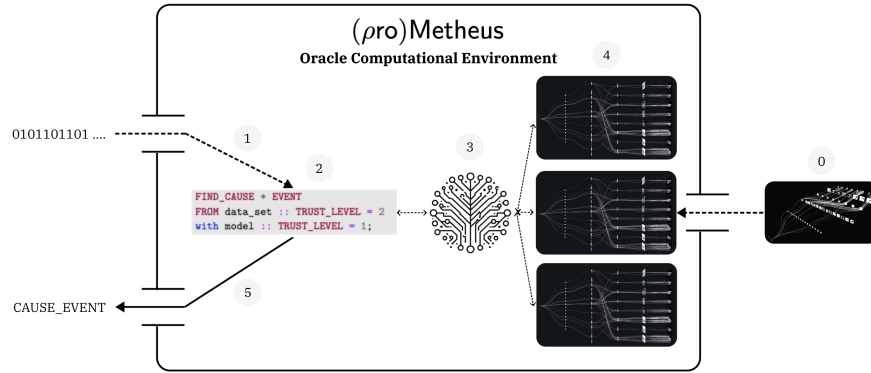
1.1 Technical Stakeholder Perspective

This is a system specification document for the (ρ ro)Metheus **Provenance Resolution Oracle** for Covenant-Bound Artificial Intelligence. It features:

- A *regulator-compliant* computational environment that enables analysts to run *audited* machine learning (ML) models on *validated* datasets.² We refer this set of ML models and datasets as *computational assets*.
- The (ρ ro)Metheus oracle is especially suitable for recovering casual relationships in datasets in a statistically rigorous manner.
- The theoretical underpinnings of the (ρ ro)Metheus Oracle is a domain specific programming language named: Typed ρ -Calculus for covenant-bound probabilistic causal intelligence. It combines decades of research and design in programming language theory, classical statistics, and recent advances in machine learning to produce a safer way of interacting with existing AI agents.

1.1.1 Computational Asset Lifecycle in (ρ ro)Metheus

In concrete terms, (ρ ro)Metheus is a user interface targeting industry professionals at banks, insurance companies, and regulators.



This user interface is the visible face of the (ρ ro)Metheus protected computational environment, whereby analysts interact with audited computational assets. The asset lifecycle proceeds as follows:

- (0,1): model and data are audited using stakeholder-defined rules of validation, and subsequently assigned a type, or more precisely a Covenant-Judgement. These legally-tagged computational assets then enter the (ρ ro)Metheus environment.
- (2) analysts query the data lake with Typed ρ -Calculus in lieu of an LLM-based chat assistant. In particular, they query data with the intention of computing the cause of some real-life **EVENT**.
- (3) (ρ ro)Metheus compiles the Typed ρ -Calculus language and mobilizes legally-compliant class of data and machine learning models to complete the task.
- (4) (ρ ro)Metheus draws on open-source and bespoke ML models to compute the likelihood of different causes of the **EVENT**.
- (5) (ρ ro)Metheus returns probable causes of this **EVENT** to the analyst.

² See definition 1 for more on computational environment.

In the figure above, the programming language Typed ρ -Calculus is the top-level user interface. Alternatively we may explore other user interfaces such as:

- A graphical user interface, which may be more appropriate in a consumer setting.
- A natural language interface using LLM's code-generation feature. In this case, we train an LLM to learn a mapping from natural language onto Typed ρ -Calculus.

The advantage of (ρ ro)Metheus computational environment is three-fold:

- All data and models are audited in accordance to stakeholder-defined laws of validity and security.
- These rule governing validity of information are then propagated along Typed ρ -Calculus's type system from data collection, to model training, fine tuning, and finally inference.
- Additionally, Typed ρ -Calculus has rigorous notions of causality that cannot be expressed in the "classical statistics" underlying machine learning models. Classically trained agents such as auto-regressive models (read: LLMs) can only express correlation, not causation.

The heart of (ρ ro)Metheus Oracle is the domain specific programming language: Typed ρ -Calculus for covenant-bound probabilistic causal intelligence. Unlike classical statistic language, Typed ρ -Calculus programs expresses causation in addition to correlation.

► **Definition 1.** [Sch13] A **Computational Environment** involves the collection of computer machinery, data storage devices, work stations, software applications, and networks that support the processing and exchange of electronic information demanded by the software solution.

Next we expand on two use cases of (ρ ro)Metheus in particular:

- one that targets who query audited data using audited ML models and validated datasets;
- one that targets regulators who define the criteria of valid data and models.

Although they appear to be two orthogonal deployment scenarios, they are in fact part of the same pipeline that takes regulator's legal requirements, and translate them into a compliant artificial intelligence computational environment.

1.1.2 An Example for Analysts

The analysts at banks and insurance companies interact with (ρ ro)Metheus as a simplified programming language. The language of interface is Typed ρ -Calculus. It is akin to MySQL, however instead of rudimentary operations that simply query the database with:

```
SELECT *
FROM employees;
```

The Typed ρ -Calculus under proposal enables analysts to interact with their database in a categorically more sophisticated manner, by seeking the cause of events with:

```
FIND_CAUSE * EVENT
FROM data_set :: TRUST_LEVEL = 2
with model :: TRUST_LEVEL = 1;
```

In this code block, we assume `data_set` and `model` are assets that are pre-loaded in the (pro)Metheus computational environment. Some additional commentary follows:

1. (line 1): note instead of merely selecting data as is the case with MySQL, the analyst uses Typed ρ -Calculus to query the database for the probable cause of `EVENT`.
2. (line 2): the setting `TRUST_LEVEL=2` in line 2 expresses the trustworthiness of the data used to determine the cost of `EVENT` in line 1. In this case `TRUST_LEVEL=2` is a *regulator*-defined trust setting on the validity of the data under consideration. It is effectively a filter on the quality of data used to determine the final cause of `EVENT`.
3. (line 3): the statistical models are also typed with `TRUST_LEVEL=1`, which determines the quality of statistical model used to determine the final cause of `EVENT`. This is critical as the quality of statistical models is a function of:
 - the quality of statisticians training them;
 - the rigor of the assumptions underlying the models;
 - and most importantly, the quality of data the model is trained on.

Naturally models trained on `TRUST_LEVEL=2` data can do no better than attaining a `TRUST_LEVEL=2` itself, as the adage: "garbage in garbage out" attests.³ The proper propagation of `type` assignment from datasets to models trained on said datasets is a core feature of Typed ρ -Calculus. Now observe that in the Typed ρ -Calculus query above, the quality of dataset is lower than the quality of the model. Then intuitively the final `CAUSE` of `EVENT` determined by this short program should carry `TRUST_LEVEL=2`.

The interaction of datasets and models at inference time is an example of *program composition*. In programming language (PL) parlance, the *inferred type* of the final recommendation correspond to the confidence of the final recommendation in statistical terms. The intellectual core of Typed ρ -Calculus pivots upon this correspondence between type in the PL sense, and a promise or *covenant* of compliance in the legal sense. We term this category of legally-compliant and statically guaranteed artificial intelligence: Covenant-Bound Artificial Intelligence. This correspondence is explored in table (1).

The intellectual core of Typed ρ -Calculus pivots upon the one-to-one correspondence of *covenant* in the legal sense, with inferred the *type* of data in the programming language sense.

1.1.3 An Example for Regulators

In the previous example, we assumed the criteria that determined which computational asset satisfies any particular `TRUST_LEVEL` was a given. In reality, this criteria is determined by regulators. This section provides an example of how regulators would interact with (pro)Metheus to define the vectors

³ For the sake of simplicity we assume `TRUST_LEVEL=2` is less trustworthy than `TRUST_LEVEL=1`.

of quality along which computational assets are audited against. Since a regulatory environment with broad stakeholder buy-in is seldomly unilaterally defined, finding consensus on the *criteria to audit* must be an interactive process. Referencing fig (1), $(\rho\text{ro})\text{Metheus}$ presents a user interface whereby:

1. Multiple parties enter a common covenant proposal environment, so as to submit candidate criteria for community judgement.
2. What follows is an iterative process, whereby the stakeholders debate and elect the most sensible set of criteria to judge data validity and AI model quality.
3. Finally this criteria is reified in the $(\rho\text{ro})\text{Metheus}$ computational environment as a type or Covenant-Judgement.

One example of such a Covenant-Judgement is **TRUST_LEVEL** that will be used by the analysts in the example above. Finally observe that depending on the quality of the dataset or AI model, it may only witness some of the criteria defined in Covenant-Judgement. This fine-grained satisfaction is what give rise to different levels of **TRUST_LEVEL**, i.e. **TRUST_LEVEL** = 1 etc.

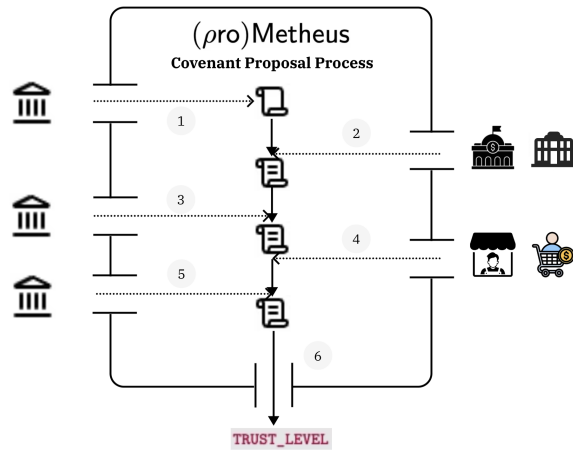


Figure 1 The $(\rho\text{ro})\text{Metheus}$ covenant proposal process is an iterative procedure where the regulator proposes criteria of audit in step (1). In step (2), other stakeholders such as financial institutions amend the proposal. This is sent to the regulator for review in step (3). Then more stakeholders are invited to the roundtable for review in step (4). In step (5), regulators give final approval. And in step (6), this law is translated into code as part of the Typed ρ -Calculus Covenant-Judgement system. This community-defined type or Covenant-Judgement can now be used to audit AI models and datasets.

$(\rho\text{ro})\text{Metheus}$ replace the notion of "code is law" with "**type is law**." Here the subjects under legal audit are computer programs, not people.

The most important aspect of this Covenant-Judgement construction process is that there is a one-to-one correspondence between drafting and enforcing regulations, and designing types and type-checking code. This is $(\rho\text{ro})\text{Metheus}$ ' interpretation of the adage "code is law," or alternatively the *Covenant-Type Correspondence*. This correspondence is outlined in table 1.

1.1.4 An Example for Policy Holders

In addition to a regulator and analyst facing role, the $(\rho\text{ro})\text{Metheus}$ Causal-Intelligence Oracle also has an insurance policy-holder facing application. This application is targeted towards small and

Law	Type
Assign Legal Status	Type Judgment or Covenant-Judgement
Drafting Legislation	Type Design
Core Constitution	Type Primitives
Legislation Amendment	Type Extension
Legislation Enforcement	Type Checking by Compiler
Upholding Covenant	Does Type check
Violating Covenant	Does not Type Check
Relationship before the Covenant	Data/Model Composition before the Compiler
Proving Action is Legal under Terms of Covenant	Proving Typed ρ -Calculus Maintain Logical Invariants

■ **Table 1** The correspondences between legal covenants and type system in Typed ρ -Calculus is the theoretical underpinnings that give a *computational interpretation* to the notion that *compiling code is the same as applying the law*. Under this correspondence, when the (pro)Metheus compiler type checks the Typed ρ -Calculus code, it is also auditing the AI agents and datasets for legal compliance.

medium businesses (SMEs) in select transnational supply chains. In this game, a parcel moves across multiple SMEs, each one of which insured by (pro)Metheus or one of its partners. In the event where the parcel is damaged or not delivered, (pro)Metheus Causal-Intelligence has to identify the party at fault. We structure this process as an *adjudication game*. There are two possible adjudication regimes: the interactive and the Causal-Intelligence automated process. Referencing fig (2):

- **Policyholder Mutual Accountability Interactive Application:** in this arrangement, each stakeholder or policyholder along the supply chain interacts with a natural-language-based dialogue-based system. They proceed in a two stages:

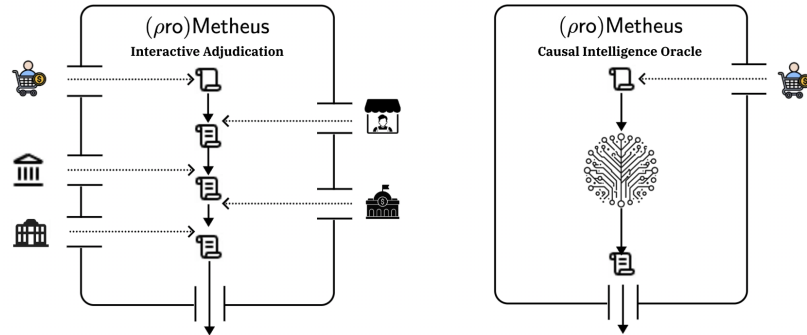
1. **(Defend):** in stage 1, each party may provide additional data proving the condition of the product before and after their touch point, so as to defend their innocence.
2. **(Accuse):** in stage 2, the party may provide evidence, or plausible explanation - stating that another member(s) along the same supply chain is liable.

Finally, a certified examiner will perform a final review of the information and decide on an outcome. Payouts will be allocated according to defined policies, and the associated parties' insurance premium will be adjusted appropriately. The interactive setting is also called the *expert demonstration* loop, and it accomplishes four objectives:

1. **(Operational leverage):** is gained in the adjudication process, as policyholders are partaking in the investigation process.
2. **(Mutual accountability):** is enculturated, where various parties along the supply chain hold each other accountable for the quality of the parcel.
3. **(Community policy retention):** each member along the supply chain are part of the same co-op, and have shared responsibility to deliver the product unmarred. In the event where "bad" events occur, their voice is heard as they are given a chance to prove their innocence.
4. **(Training data):** the interactive adjudication process is also a data labeling process. It will be used to train the (pro)Metheus Causal-Intelligence Oracles, enabling automation.

- **The (pro)Metheus Causal-Intelligence Automated Oracle:** in this setting, the (pro)Metheus Oracle validates the parcel history, and decides on an outcome based on its internal Causal-Intelligence engine. This process is legally compliant as the (pro)Metheus program judging the claim is

expressed in Typed ρ -Calculus, so that each step of its computation graph observes the necessary legal regulations.



■ **Figure 2** (Left): the $(\rho\text{ro})\text{Metheus}$ interactive adjudication application is a *mutual accountability game*, whereby each policy holder is asked to defend their role in the supply chain by submitting additional data proving their innocence, and then accuse a neighbor in the chain of malpractice that degrades the final quality of the parcel. Finally a certified examiner passes judgement and closes the case with appropriate payouts and adjustments. (Right): the $(\rho\text{ro})\text{Metheus}$ Causal-Intelligence automated adjudication Oracle ingests the claim along with data generated by the underlying commodity flow, and decide on the outcome directly sans mandatory human input.

The $(\rho\text{ro})\text{Metheus}$ Causal-Intelligence Oracle grants immense operational leverage and risk assessment abilities. Thus allowing us to **insure the uninsurable**.

1.2 Broad Stakeholder Perspective

The technical perspective may be sufficient for those with a background in both machine learning and programming language theory, however it is too abstract for the non-technical audience. This section pulls the lens back and recontextualize $(\rho\text{ro})\text{Metheus}$ within the technical and political milieu of the present moment. We answer the question: why is $(\rho\text{ro})\text{Metheus}$ necessary when a diverse set of tools exists to characterize the confidence, validity, and scope of statistical models and datasets?

Why $(\rho\text{ro})\text{Metheus}$

The demand arises with the latest tranche of breakthroughs in AI around large language models (LLMs). Nominally LLMs are trained via regression on time series data, therefore they are not categorically different from i.e., deep convolution neural networks (CNNs) trained on non-time-series data, or a soft-object manipulations algorithm sampled via reinforcement learning. However the *user-behavior* that have spawned around LLMs applications differ significantly from earlier machine learning tools. Presently users are not only using LLMs as general purpose information retrieval systems, but they are also prompting them as "intelligent reasoning" engines. This user behavior spans across many settings:

- *Financial setting*: banks and insurance companies are using LLMs in high-valued contexts such as producing analyst reports to inform buy/sell decisions. They have never used CNNs in this way. Moreover, the prior generation of financial models used to price assets are either simple, i.e. linear regression on a few factors, or reasonably explainable models based on heat-diffusion equations. In contrast, LLMs are complex and opaque.
- *Medical setting*: doctors are using LLMs to summarize patients' past to inform an opinion, while patients are using LLMs to diagnose their illness. This has far reaching consequences extending past the hospital setting, including the pricing of insurance and other ancillary services.
- *Consumer setting*: people are treating LLMs as friends, teachers, and personal psychologists. This use case may appear the most trivial, however social media has already demonstrated its capacity to silo citizens into misinformation echo chambers and throw elections. Moreover, whereas the harm due to social media is bounded by the number of people who posts, LLM-based social media tools have no such upper bound. This presents a long-term challenge to the health of any society.

In summary, unlike prior deep learning models, LLMs function as "artificial people," or more to the point: "entry level employees." Now given the variety and sensitivity of LLMs' application domains, we assert LLM-based applications require stricter oversight, perhaps even legal oversight, from regulators beyond rudimentary internet guidelines.

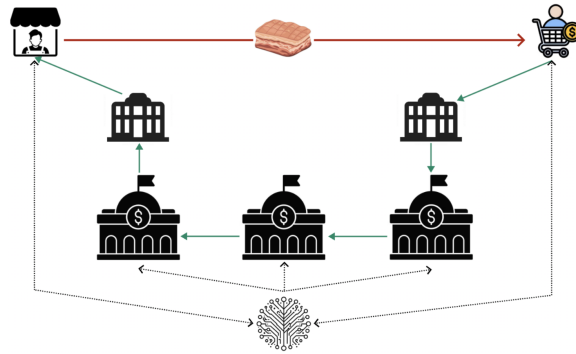
1.2.1 The (pro)Metheus Oracle Desiderata: a High Level Attempt

The (pro)Metheus computational environment is a protected sandbox with strictly defined and enforced rules. In particular, the *type-system* of the (pro)Metheus Typed ρ -Calculus is crafted to satisfy the following desiderata:

1. *Flexibility*: regulators from across jurisdictions may define laws governing the behavior of LLMs in a way that suit their local conditions. This is important since language models summarize the internet and with it: culture. There cannot be one set of universal "LLM-laws," as different cultures carry different standards of "good behavior." Therefore the defined boundary conditions must be different.
2. *Composability*: it would be prohibitively expensive, not to mention tedious, to define laws governing every invocation of an AI agent. Thus Typed ρ -Calculus language is defined so that regulators can specify simple rules, and when composed they form a rich system spanning the behavior of an AI agent.
3. *End-to-end oversight*: once the boundary conditions are defined, regulator-privilege extends deep into any AI-based system over the course of its application lifecycle. This means regulators may define high-level laws governing what is good AI behavior. Then the laws would propagate from data collation, to model-training, refinement, and finally inference as well as post-inference data-collection steps. If one conceptualizes an AI agent as a living being, then (pro)Metheus regulates agent-behavior from cradle to grave.

In conclusion, the (pro)Metheus Typed ρ -Calculus computational framework operationalizes the adage "code is law." In this case the subjects under legal audit are not people but computer programs. In more technical terms:

The (pro)Metheus Typed ρ -Calculus language is a new way of realizing *code as law*. It does so with a novel application of type systems common in statically-typed programming languages. Whereby AI agents and data constitutes the term universe, while the regulator legal framework defines the type universe. This type system, or legal framework, governs:



■ **Figure 3** The (ρ) Metheus oracle applied to supply chain orchestration, whereby vendors sell foodstuff to buyers across national borders. The (ρ) Metheus oracle informs trade across stakeholders along this transnational network servicing: buyers, sellers, commercial/central banks, and insurers.

- how AI learns;
- once learned, how AI interacts with people, data, and each other.

1.2.2 The (ρ) Metheus Stakeholder Stack

The fact that (ρ) Metheus interacts with regulators at the sovereign level is a critical differentiator and unique go-to-market channel. Presently both financial and medical institutions are inundated with "AI offerings" from major firms and startups alike.

- The startups sell GPT wrappers: that is new way of selling the same old enterprise workflows. This game is stale and soon they shall be cannibalized.
- Major firms such as Microsoft and Google use "AI-agents" as cheese in the trap to sell what they have always sold: API calls and data egress. Ambitious managers use these projects to pad their promo-package. And once promoted, the projects are typically orphaned and whither away.

We escape this red ocean completely and appraise the stakeholder stack as follows:

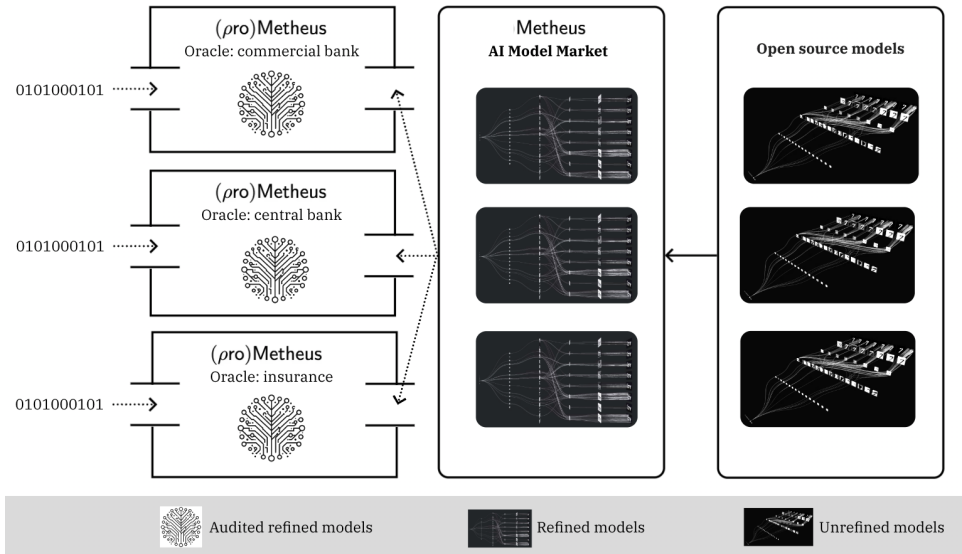
- *Sovereign regulators*: (ρ) Metheus sell protected computational environments, whereby all code execution is compliant with respect to local laws and norms. The importance of security cannot be overstressed, especially as we enter a multipolar era whereby all things on the chessboard enter a state of motion. Since laws are public goods that rightly belong in the public domain, we engage with regulators through the nonprofit foundation FairCovenant. See the whitepaper at <http://bit.ly/44EiQV0> for details on how we engage with regulators [W P25].
- *Enterprises*: (ρ) Metheus sells compliance as a service, so that the downsides of said institutions using AI are protected. This is critical: if AI is to make high valued decisions, then someone must be liable - (ρ) Metheus answers this question in a principled manner.
- *Enterprises Analysts*: (ρ) Metheus offers a high level interactive environment whereby analysts may query the data for causal relations. This dovetails how people are using LLMs anyways. However the statistical language describing this class of auto regressive models *cannot express causation*. The (ρ) Metheus Typed ρ -Calculus language fills this gap by expressing causality via a principled mathematical language.

The last point bears repetition. Recall Typed ρ -Calculus is not just for regulator-compliance, but also for *probabilistic causal reasoning*. We did not stress this aspect of (ρ) Metheus in the introduction, but it shall be specified in the main body shortly.

1.3 The Greater Metheus Ecosystem

Although (pro)Metheus is a general purpose computational oracle, it is initially deployed for stakeholders along specific supply chains to support transnational trade organizations (see figure 3). Additionally, (pro)Metheus will activate the larger open-source AI community by drawing both from freely available models, as well as incentivizing the community to fine-tune or train new models. The process is outlined in figure 4:

- *left column*: there will be separate (pro)Metheus oracle environment for each industry stakeholder. This is appropriate as each player has different standards of data validity, and AI model security. These separate requirements find congruence in the Typed ρ -Calculus type system.
- *center column*: (pro)Metheus is connected to an open source AI model market, whereby foundational models are refined by distributed teams of ML engineers. ⁴
- *right column*: this moment in technology is unique because of the gluttony of open source foundational models. We funnel this set of commodified assets into the privileged environment of (pro)Metheus oracle.



■ **Figure 4** The (pro)Metheus Oracle connects enterprises with the broader AI foundational model ecosystem by embedding refined AI models within a rigorous legal framework.

Observe that (pro)Metheus’s moat is the regulator and stakeholder-defined laws of data validity and model security. That is say: **their wall is our moat.**

In (pro)Metheus, the regulator’s wall is our moat.

⁴ See monograph at <http://bit.ly/4IFn8Sf> for details on AI model market [Lin25].

2 The Covenant-Type Correspondence

This section underlines the principled basis of Typed ρ -Calculus, that is the correspondence between:

- a legal covenant,
- and types that mediate how program assets interface with each other.

This correspondence formalizes the notion that *compiling code is the same as applying the law*; or simply "code is law." Some definitions and examples are listed below, followed by more in depth discussion on the correspondence.

In (ρ)Metheus, code that compiles is also code that obeys the law. That is to say compliant computations run to completion, while illegal computations fail *before* execution. This is know as: **static guarantee of covenant compliance**.

2.1 Preliminary Definitions and Examples

► **Definition 2.** [25a] In law, a **legal covenant** is a binding agreement or promise, often included in a contract or deed, that obligates one party to perform or refrain from a specific action. It can be 1) positive covenant requiring an action, or 2) a negative covenant restricting an action.

► **Example 3.** [25b] In (Property Law), there are two types of restrictive covenants: affirmative and negative:

- An affirmative covenant obligates a person to act. For example, a covenant that requires the homeowner to keep the trees trimmed in the yard is an affirmative covenant.
- A negative covenant prohibits a person to act. For example, a negative covenant can forbid a homeowner from building a fence.

► **Definition 4.** [Pie02] A **Type System** is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute. Its theoretical basis lies in early 20th century study of logic, mathematics, and philosophy. In computational terms, programs equipped with types allows the software engineer and the compiler to reason about the run-time behavior of the program. ⁵

► **Example 5.** (**Int** and **String**) In statically typed languages such as **Haskell**, every data and function in the language is associated with a type. The type specifies the proper behavior the function, while the compiler ensures the program type-checks, or in more informal terms: "behaviorally correct" *before* the code runs. This removes entire class of errors from run time and ensure the code does not behave poorly as it executes. The following code block defines a function **strlen** counts the length of a **String**, in this case written as [**Char**].

⁵ A compiler is a software program that translates one programming language to another.

```

strLen :: [Char] -> Int
strLen [] = 0
strLen x:xs = 1 + strLen xs

```

Observe this function is typed so that it can only accept a string (in this case represented as list of **Characters**), so that we have: `strLen "hi" = 2`. If instead the user passes a **Boolean** value into the function with `StrLen true`, then the function `strLen` will fail outright since it is "contractually obligated" to only accept values of type `[Char]`.

2.2 The Correspondence

The type-covenant correspondence occurs in two phases: 1) the drafting of legislation or type system design, and 2) the enforcement of legislation or static type checking.⁶ A summary of the similarities is found in table 2.

Law	Type
Assign Legal Status	Type Judgment
Drafting Legislation	Type Design
Core Constitution	Type Primitives
Legislation Amendment	Type Extension
Legislation Enforcement	Type Checking by Compiler
Upholding Covenant	Does Type check
Violating Covenant	Does not Type Check
Relationship before the Covenant	Data/Model Composition before the Compiler
Proving Action is Legal under Terms of Covenant	Proving Typed ρ -Calculus Maintain Logical Invariants

■ **Table 2** The correspondences between legal covenants and type system in Typed ρ -Calculus is the theoretical underpinnings that give a *computational interpretation* to the notion that *compiling code is the same as applying the law*. Under this correspondence, when the Typed ρ -Calculus compiler type checks the Typed ρ -Calculus code, it is also auditing the code for legal compliance.

Now we expand table 2 as follows:

- *Legislation design*: the drafting of legislation corresponds to designing type system within Typed ρ -Calculus.
 1. In real life one finds a core constitution, with many extensions that flow from the core structure. Similarly within the (pro)Metheus computational environment, one will find primitive type and semantics that define entities and their relations. This is followed by extensions defined by the non-profit foundation defined in [W P25].
 2. In particular, in (pro)Metheus Typed ρ -Calculus, the correspondence between type and covenant arises in the negative direction. That is to say the Typed ρ -Calculus compiler prevent non-compliant AI agents from running.

⁶ Normally we abhor the egregious anthropomorphization of machine learning in popular literature. But in this setting there is a fine parallelism of how human societies relate to the *legal machinery*, and how AI agents in (pro)Metheus computational environment relate to the Typed ρ -Calculus type system. That is to say in both cases we focus on the rationalization or mechanization of relationships.

3. Finally just as covenants govern the behavior of people w.r.t each other before the state, in (pro)Metheus the type system governs the *composition* of data with data, data with models, and model-to-model composition.
- *Legislation enforcement*: unlike covenants in the context of people, inside the (pro)Metheus computational environment, AI agents and associated software cannot violate covenants at all. This is because as Typed ρ -Calculus mobilize computational assets such as data and AI agents, said assets are type-checked before they are used in regulated computations. The advantage of AI agents over people is that while people may violate covenants, digital covenants defined by Typed ρ -Calculus logic are absolute: no AI agents may break the type system provided they are bound to (pro)Metheus. Therefore compliant computations run to completion, while illegal computations fail *before* execution. This property of the (pro)Metheus environment is called *static guarantee of covenant fulfillment*.

3 Formal Specification of the (pro)Metheus Covenant

3.1 Introduction

The (pro)Metheus Typed ρ -Calculus language is an example of a domain-specific programming language or DSL. DSLs are common in industry, and span from simple scripting languages for ad-hoc tasks, to complex languages embedded into a far-reaching ecosystems. The primary example here is MatLab. A complete (pro)Metheus computational environment is comparable to MatLab in complexity. However whereas MatLab is a tool with low level integration with the legal structure of academia, (pro)Metheus is deeply embedded into the financial and legal aspects of transnational trade systems. This requires a broad surface area of engagements with legal, financial, and insurance entities servicing said trade routes.

Consequently, the determination of core type or covenant primitives in the Typed ρ -Calculus DSL is part of a broader governance process of The FairCovenant Foundation outlined in [W P25]. In particular, FairCovenant collaborate closely with stakeholders to determine the proper regulatory requirements needed to deploy AI agents. These requirements are then encoded into the type system, or legal system of (pro)Metheus computational environment. See figure 5 for how the process is denoted procedurally.⁷

Moreover, since the type system is shared by multiple stakeholders that span many jurisdictions, the type system itself exists as a *public good*, and therefore is financed as such. Since the exact design of the core type system is delegated to the FairCovenant Foundation, the rest of the paper will speak of types in the generic sense with symbol κ . Covenant type assignment is then written with:

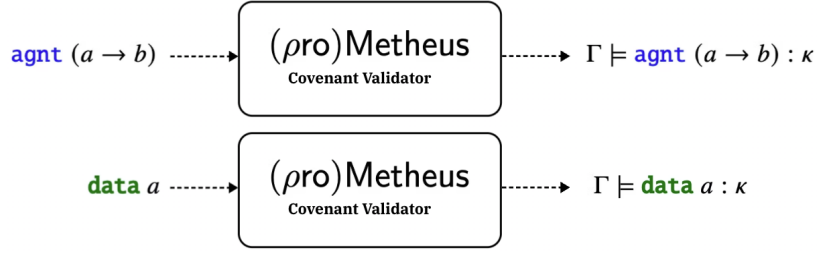
$$\Gamma \models \text{agnt} : \kappa_1 \quad \Gamma \models \text{data} : \kappa_2. \quad (1)$$

Expression 1 is read: under the *validation environment* Γ defined by The FairCovenant Foundation, the AI **agnt** satisfies covenant κ_1 , and the **data** has covenant value κ_2 . The symbol Γ is also referred to as "typing context" in programming language terms. See section 4.1 for more on type judgement.

3.2 Prior Art

Now we reify the type assignment of expression (1) with concrete illustrations. Over the past few years, there has been a growing call to better document machine learning models and the data they are

⁷ Link to whitepaper: <http://bit.ly/44EiQV0>



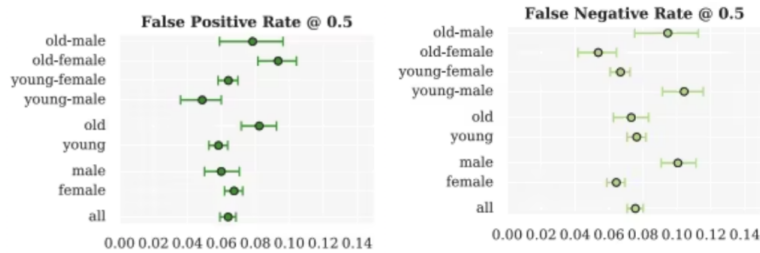
■ **Figure 5** The (ρ)Metheus covenant-validator Γ ingests **data** or **agnt**, and assigns some Covenant-Judgement κ to each computational asset. The exact rules governing the content of the covenant κ is determined by The FairCovenant Foundation, so that once judged the **agnt/data** is deemed regulator-compliant. This κ now governs how said computational asset can interact with other assets in the (ρ)Metheus protected environment. Here compliant computations run to completion, while illegal computations fail *before* code execution. Thus illegal code cannot even run. This property is called: *static guarantee of covenant fulfillment*.

trained on. The next two examples represent preliminary attempts of assigning types to AI assets. The (ρ)Metheus Typed ρ -Calculus ecosystem will take the flavor of documentation presented in examples 6 and 7, and express them as types.

► **Example 6.** [M M19] (**Model Cards**) are documentations (or meta-data) that determines the application scope and provenance of machine learning models. This is an example of $\Gamma \models \text{agnt} : \kappa$. From the abstract the authors state:

“Model cards are short documents accompanying trained machine learning models that provide benchmarked evaluation in a variety of conditions...that are relevant to the intended application domains. Model cards also disclose the context in which models are intended to be used, details of the performance evaluation procedures, and other relevant information... this framework can be used to document any trained machine learning model.”

An example of documentation for a particular model is presented in figure 6.



■ **Figure 6** In this model for smiling detection in images reported in [M M19]. Here the false positive and false negative rates for each model is reported, broken down by demographic data. Documenting the performance envelope of a model is important if the model is to be deployed in sensitive areas, such as detecting the likelihood of a criminal from images. This likelihood could be used to make arrests, which presents a challenge as said estimations are often skewed by the exiting on criminals. In (ρ)Metheus Typed ρ -Calculus environment, the totality of a given model’s metadata would be abstracted into a unique type $\mathbf{T}$.

► **Example 7.** [Mah22] (**Data Cards**). Similar to model cards, data cards document the provenance, validity, and applicability of data used to train machine learning models. This is an example of $\Gamma \models \text{data} : \kappa$. From the abstract we have:

“we propose Data Cards for fostering transparent, purposeful and human-centered documentation

Section Title			Row 1	Section
Dataset Overview				
A	DATASET SUBJECT	DATASET SNAPSHOT	DESCRIPTION OF CONTENT	
B	Bold to select all applicable. Do not delete any unselected choices.	Fill out details as indicated, adding rows as needed. If a requested detail is inapplicable, following guidance on N/A. Include links to additional table(s) with more detailed breakdowns in the caption.	Provide a short summary of the dataset content. Include links where applicable. E.g. bounding-box annotations and labels in images of coarse-and fine-grained objects.	
C	Sensitive Data about people	Size of dataset	123456 MB	
	Non-Sensitive Data about people	Number of Instances	123456	
	Data about natural phenomena	Number of Fields	123456	
	Data about places and objects	Labeled Classes	123456	
	Synthetically generated data	Number of Labels	123456789	
	Data about systems or products and their behaviors	Average labels per instance	123456	
Unknown	Algorithmic Labels	123456789		
Others*	Human Labels	123456789		
(*please specify)	Other	123456		
		+write here+		
Blocks			Row 2	
DESCRIPTIVE STATISTICS				
Add basic statistics for each field here, as relevant. If there is insufficient space, focus on the top seven most important or critical fields for this dataset. E.g., some statistics will be relevant for numeric data, but not for strings.				

■ **Figure 7** Similar to model cards, data cards document meta-information on the dataset used to train AI models. Since machine "learning" is data compression, this meta information is then appropriately inherited by any AI model trained on the data, along with any additional meta-information accumulated while the model is under training. Image from [Mah22].

of datasets within the practical contexts of industry and research. Data Cards are structured summaries of essential facts about various aspects of ML datasets needed by stakeholders across a dataset’s lifecycle for responsible AI development. These summaries provide explanations of processes and rationales that shape the data and consequently the models—such as upstream sources, data collection and annotation methods; training and evaluation methods, intended use; or decisions affecting model performance.” An example data card format is show in figure 7.

The cited paper [M M19] on model cards is not just academically forward, the careers of the authors are also cautionary tales for those who push for regulation on AI agents without proper institutional support. In particular, the authors Margaret Mitchell and Timnit Gebru are of an "activist mindset." They would later author a different paper challenging the ability of deep learning models to think. No serious scholar in the field confuses deep learning model inference for "thinking." However the conduct of the authors nonetheless drew the ire of management at Google. They found themselves in the crosshair of a Distinguished Google Fellow, the two women were subsequently fired abruptly.



MMitchell
@mmitchell_ai

"didn't meet our bar for publication ... It ignored too much relevant research".
He says of our paper, with over 128 cross-discipline citations. And so fires one of the authors. 2020's monster version of Reviewer #2.



Jeff Dean
@JeffDean · Dec 4, 2020

I understand the concern over Timnit's resignation from Google. She's done a great deal to move the field forward with her research. I wanted to share the email I sent to Google Research and some thoughts on our research process.

docs.google.com/document/d/1f2...

3:24 AM · Dec 6, 2020 from Los Angeles, CA

Example 6 is a case study of how *not* to push for closer scrutiny on AI models, especially when so many livelihoods, reputations, money, and egos are on the line. This is why (pro)Metheus deploy AI regulation and legal framework through the nonprofit foundation FairCovenant. Only by working closely with regulators, insurers, and central banks in their domain of operation, can we externalize the necessary *legitimacy* to regulate artificial intelligence.

3.3 Formal Specification of the (pro)Metheus Covenant

This section formally specifies the (pro)Metheus Covenant as a privileged datatype in the Typed ρ -Calculus universe. Recall the details of the data validation criteria are designed and maintained by The FairCovenant Foundation. The format of said validation criteria is akin to a checklist. In the event whereby a criterion is real valued, then it can be discretized into regions. This region is then expressed as a binary tree, so that a real value falling within one of the zones correspond to a particular path down the tree.⁸

► **Definition 8.** (An *agnt* or *data* validity criteria C) is a check-list of quality measures that the *agnt* or *data* must satisfy to partake in the (pro)Metheus computational environment.

The data and model cards of examples 7 and 6 are instances of validity criteria C .

► **Definition 9.** (A Covenant-Judgement κ). For every set of *agnt* or *data* validity criteria C of length $|C| = n$ defined by The FairCovenant Foundation, so that each criteria in C is satisfied independently of another. Now let the (pro)Metheus Oracle judges some *agnt* or *data* according to this criteria C , so that it generates a verdict $\kappa_{2^n, i}$ of length n , whereby:

- $\kappa_{n, i}[j] = 1$ if the *agnt* or *data* satisfies criterion j .
- $\kappa_{n, i}[j] = 0$ otherwise.

This n -bit vector $\kappa_{n, i}$ is a Covenant-Judgement w.r.t the criteria C . Moreover, the following three statements are equivalent:

- *agnt* or *data* has Covenant-Judgement $\kappa_{n, i}$;
- *agnt* or *data* witnesses $\kappa_{n, i}$.
- In formal notation:

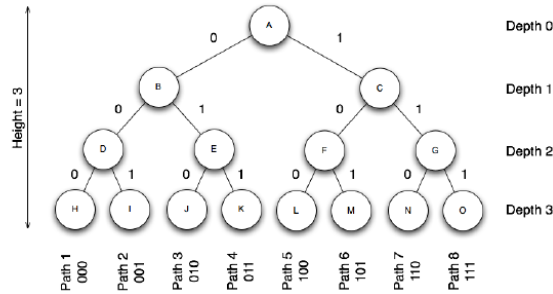
$$\Gamma \models \text{agnt} : \kappa_{n, i} \quad \text{or} \quad \Gamma \models \text{data} : \kappa_{n, i}.$$

We denote the set of all possible covenants generated by criteria C with the symbol $\mathbb{K}$. Observe that given a criteria set of length n , $\mathbb{K}$ is exactly the set of all paths down a binary tree of height n :

$$\mathbb{K}(C) = \{\kappa_{n, 0}, \dots, \kappa_{n, 2^n - 1}\}.$$

And each Covenant-Judgement $\kappa_{n, i}$ is one path down this tree (see figure 8).

► **Example 10.** (A covenant set $\mathbb{K}(C)$) over three criteria is the set of paths down a binary tree of depth 3, and may be drawn:



■ **Figure 8** A binary tree of depth 3 induces $2^3 = 8$ possible 3-bit strings. Each string is an example of a covenant indexed by 3, written: $\kappa_{3, i}$.

⁸ This formulation is elegant because the covenant itself is also structured as a binary tree, see definition 9.

In this example, we may enumerate the possible Covenant-Judgements with:

$$\mathbb{K}(C) = \{\kappa_{3,0}, \kappa_{3,1}, \dots, \kappa_{3,7}\} \text{ where}$$

$$\kappa_{3,0} = 000 \quad \kappa_{3,1} = 100 \quad \kappa_{3,2} = 010 \quad \kappa_{3,3} = 001 \quad \kappa_{3,4} = 110 \quad \kappa_{3,5} = 101 \quad \kappa_{3,6} = 011 \quad \kappa_{3,7} = 111,$$

so that each $\kappa_{3,x}$ is a particular Covenant-Judgement. For example if a particular **data** a satisfies the first and third criteria only, then **data** a witnesses 101. Its Covenant-Judgement is written:

$$\Gamma \models \text{data } a : \kappa_{3,5}, \text{ where } \kappa_{3,5} = 101.$$

► **Remark 11. (Simplified notation on Covenant-Judgement).** The notation $\kappa_{3,1}$ is cumbersome, so we prefer to write κ_1 instead, and leave the length of the bit-string 3 implicit.

Additionally, Typed ρ -Calculus must judge covenants by comparing them numerically, so that $(\rho\text{ro})\text{Metheus}$ may price covenant witnesses accordingly. Thus we will define a notion of distance and "less than." Hence we place an algebra on $\mathbb{K}$ as follows.

► **Definition 12. (A covenant space $\mathbf{K}$)** with respect to some FairCovenant Foundation defined criteria C is a metric space equipped with a distance function. Written:

$$\mathbf{K}(C) = (\mathbb{K}(C), ||), \text{ where } || : \mathbb{K} \times \mathbb{K} \rightarrow \mathbb{R}. \quad (2)$$

Where the distance function $||$ is defined by a suitable arithmetization of the Covenant-Judgements in $\mathbb{K}$. This arithmetization is defined by The FairCovenant Foundation, so that given arithmetization function arith , we have:

$$\kappa_i || \kappa_j = \text{arith}(\kappa_i) - \text{arith}(\kappa_j). \quad (3)$$

From this point forward, when we say "covenant," we refer to the covenant space $\mathbf{K}(C)$, so that for every FairCovenant criteria of length n , there is an associated covenant space $\mathbf{K}(C)$. Now we consider the issue of Covenant-Judgement composition. This arises in settings when:

- Datasets are composed with other datasets, and they have different covenant criteria. For example, **data** a may be a set of images, while **data** b may be a set of image-aligned captions. They will satisfy different covenant criteria, however the combined dataset⁹ will carry Covenant-Judgement information from the source datasets.
- Trained **agent** that witnesses some Covenant-Judgement κ is refined on **data** -set of Covenant-Judgement κ' . Then the resultant model must have some type κ'' , the content of this κ'' must be defined.
- The output of one **agent** ($a \rightarrow b$) is input some other **agent** ($b \rightarrow c$). If **agent** ($a \rightarrow b$) witness κ , while **agent** ($b \rightarrow c$) witness κ' . Then what is the resultant value c output by the model?

► **Definition 13. (Covenant Composition)** between two covenant spaces $\mathbf{K}$ and $\mathbf{K}'$ generated by two different criteria C and C' , is the Cartesian product of the two spaces. Written:

$$\mathbf{K} \otimes \mathbf{K}' = \left(\left\{ \kappa_m = (\kappa_i, \kappa_j) : \kappa_i \in \mathbf{K}, \kappa_j \in \mathbf{K}' \right\}, || \right). \quad (4)$$

Now we can define the distance $||$ between two product covenants by arithmetizing each of the elements in the vector with:

$$\kappa_m = (\text{arith}(\kappa_i), \text{arith}(\kappa_j)). \quad (5)$$

⁹ Sometimes called a multi-modal dataset.

Thereby interpreting this κ_m as a real-valued vector. Then the distance function is defined as the dot product between the two vectors:

$$\kappa_m \parallel \kappa_n = \kappa_m \cdot \kappa_n. \quad (6)$$

Some further observations follow:

- Covenants are closed under composition. That is to say the product of covenants are a covenant themselves.
- Covenant composition is not necessarily commutative. That is to say we should not expect this to be true: $(\kappa_i, \kappa_j) = (\kappa_j, \kappa_i)$.
- We may introduce an identity covenant for every Foundation-defined covenant criteria, by defining this identity as the all-ones vector of suitable length. In vector space language, this corresponds to lifting some sub-space into the affine-space.

One final definition will round out the chapter.

► **Definition 14.** (*The (pro)Metheus Covenant Universe $\mathfrak{R}$*) is generated as follows:

1. Given the set of all possible criteria $C, \dots$ The FairCovenant Foundation defines, we generate the set of all possible covenant spaces with:

$$\mathbb{S} = \left\{ \mathbf{K}(C_i), \dots \mid C_i \in \{C_1, \dots\} \right\}.$$

2. Now construct the sigma-algebra over $\mathbb{S}$ with $\sigma(\mathbb{S})$.
3. And complete each subset $\mathbb{S}$ in $\sigma(\mathbb{S})$ under $\otimes$ with:

$$\mathfrak{R} = \left\{ \otimes_{\mathbf{K}(C_i) \in \Sigma} \mathbf{K}(C_i) \mid \Sigma \in \mathbb{S} \right\}. \quad (7)$$

In the remainder of the paper, we define what it means to assign every program in Typed ρ -Calculus some Covenant-Judgement in $\mathfrak{R}$. This concludes the discussion on the formal specification of Covenant-Judgements. In summary:

1. Covenants are specified by The FairCovenant Foundation, it is a list of criterion that **agnt** and **data** must satisfy to partake in the (pro)Metheus computational environment.
2. A particular instance of a Covenant-Judgement is just a bit string, which when appropriately arithmetized, becomes a real number that can be subtracted.
3. When a computational asset such as **agnt** witnesses some Covenant-Judgement κ , we assign the **agnt** with this Covenant-Judgement with the notation: $\Gamma \models \mathbf{agnt} : \kappa$.
4. When computational assets compose, the Covenant-Judgements also compose accordingly. Composition occurs when:
 - **data** compose with other **data** to augment information content.
 - **agnt** compose with **data** during training or inference.
 - **agnt** compose with other **agnt** when the output of one **agnt** is used as input into another **agnt**.

Composition allows us to build complex Covenant-Judgements, or *legal obligations* from atomic Covenant-Judgement definitions. This greatly augment (pro)Metheus' ability to audit **data** and **agnts**. The FairCovenant Foundation member's time is valuable, composition allows them to define primitive obligations that code must satisfy, so that the these covenant-primitives *span* a set of acceptable AI behavior in the (pro)Metheus universe.

4 Formal Specification of Simply Typed ρ -Calculus

The rest of the paper recursively descends downwards, so as to define Typed ρ -Calculus for covenant-bound probabilistic causal intelligence. The language is introduced by layers as follows:

1. We begin at the highest level of abstraction, and define how models and data interact with each other in the aggregate. In particular, we are sensitive to how stakeholders in the FairCovenant Foundation defines how AI assets interact with each other.
2. Next we descend down one level, and express how Typed ρ -Calculus can be used to build a probable causal graph given information drawn from raw data and information summarized by LLM models. We then show how the type system interacts with the causal graph to properly assign legal culpability as Typed ρ -Calculus computes probable cause of an event.
3. Finally, we propose future directions whereby the type system may find coverage, for example in the computational graph of deep learning models in i.e. TensorFlow.

In this chapter, we first introduces the basic technical knowledge needed to understand Typed ρ -Calculus and Loare-Logic, and then specify layer one in point (1) above.

4.1 Technical Background

This section provides the necessary background from the field of programming languages and formal verification to understand Typed ρ -Calculus used to compute causal relations from data, and the (ρ)Metheus Loare-Logic used to generate proofs of Covenant-Compliance.

Language Syntax and Typing Relations

Given a typed programming language, its *typing relations* is an inference rule that describes how the type system assigns a type to a syntactic construction.

- These rules are applied by the type system to determine if a program is well-typed and what type the expressions have.
- If the program is well-typed, then it should satisfy certain properties such as all programs can either step forward or terminate with a value.
- Types restrict the universe of allowed programs so that all well-typed programs is "well behaved" according to the designer's desiderata [Pie02].

An example follows to demonstrate the interplay between the syntax of a language and its typing relations.

► **Example 15.** [Pie17](Simply-Typed Lambda Calculus). Given a language with syntax:

<i>expressions</i>	$e := x \mid \lambda x : \tau. e \mid e_1 e_2 \mid n \mid e_1 + e_2 \mid ()$
<i>values</i>	$n := \lambda x : \tau. e \mid n \mid ()$
<i>types</i>	$\tau := \text{Int} \mid \text{Unit} \mid \tau_1 \rightarrow \tau_2.$

We introduce a relation or judgment over typing contexts Γ , expressions e , and types τ . The judgment:

$$\Gamma \models e : \tau,$$

is read as "expression e has type τ in context Γ ." Next we write the type judgement over all expressions:

$$\begin{array}{c}
\frac{}{\Gamma \models n : \mathbf{Int}} \qquad \frac{}{\Gamma \models () : \mathbf{Unit}} \qquad \frac{\Gamma(x) = \tau}{\Gamma \models x : \tau} \\
\\
\frac{\Gamma \models e_1 : \mathbf{Int}, \Gamma \models e_2 : \mathbf{Int}}{\Gamma \models e_1 + e_2 : \mathbf{Int}} \qquad \frac{\Gamma, x : \tau \models e : \tau'}{\Gamma \models \lambda x : \tau. e : \tau \rightarrow \tau'} \qquad \frac{\Gamma \models e_1 : \tau \rightarrow \tau' \quad \Gamma \models e_2 : \tau}{\Gamma \models e_1 e_2 : \tau'}
\end{array}$$

Observe this set of typing rules are applied to each expression and determines how the Lambda Calculus expressions can be typed. Furthermore, line two shows the inductive typing rules whereby complex expressions featuring function application and addition are also typed.

The (pro)Metheus Loare-Logic replicates this design pattern, by applying FairCovenant-defined Covenant-Judgement to the set of Typed ρ -Calculus expressions defined in eqn (8).

Hoare Logic

Hoare logic is a formal system with a set of logical rules for reasoning rigorously about the correctness of computer programs. Hoare-style verification is based on the idea of a specification as a contract between the implementation of a function its clients:

- The specification consists of the precondition and a postcondition.
- The precondition is a predicate describing the condition of the code or functions relies on for correct operation. The client must fulfill this condition.
- The post condition is a predicate describing the condition of the function establishes after correct running; the client can rely on this condition being true after the call to the function.

The goal of Hoare-style verification is thus to statically prove that, given a pre-condition, a particular post-condition will hold after a block of code executes. We do so by generating a logical formula known as a *verification condition*, constructed so that if true, we know that the program behaves as expected. From a syntactic standpoint, Hoare logic is presented to the programmer as *Hoare triples*.

► **Definition 16.** [Pie17] (*A Hoare Triple*) is a claim about the state before and after executing a command. The standard notation is:

$$\{\{P\}\} \varphi \{\{Q\}\}.$$

Meaning:

- If command φ begins execution in a state satisfying assertion P ,
- and if φ eventually terminates in some final state,
- then that final state will satisfy the assertion Q .

Assertion P is called the *precondition* of the triple, and Q is the *postcondition*.¹⁰

► **Example 17.** [Pie17] (**Hoare Triple**) The following:

$$\{\{x = 0\}\} x := x + 1 \{\{x = 1\}\},$$

is a valid Hoare triple, stating that command $x := x + 1$ will transform a state in which $x = 0$ to a state in which $x = 1$.

¹⁰ We use double brackets for pre and post conditions, following [Pie17].

► **Example 18. (Hoare Logic)** applied to a programming language would proceed as follows. Given a simple language:

<i>arithmetic expressions</i>	$e := x \mid n \mid e_1 + e_2 \mid e_1 \times e_2$
<i>boolean expressions</i>	$b := \text{true} \mid \text{false} \mid e_1 < e_2$
<i>commands</i>	$c := \text{skip} \mid x := e \mid c_1; c_2.$

Its set of Hoare triples are:

$$\frac{}{\{\{P\}\} \text{ skip } \{\{Q\}\}} \quad \frac{}{\{\{P[e/x]\}\} x := e \{\{P\}\}} \\ \frac{\{\{P\}\} e_1 \{\{R\}\}, \{\{R\}\} e_2 \{\{Q\}\}}{\{\{P\}\} e_1; e_2 \{\{Q\}\}}$$

Now given any program written in the language above, we can use these Hoare rules to verify the program satisfy certain conditions laid out in Q and P .

In the context of $(\rho\text{ro})\text{Metheus}$:

- the precondition P defines the quality of data and statistical models that are input into the analysis;
- the command φ is the Typed ρ -Calculus computation that determines the output;
- while the post condition Q is quality of output that the analyst-written $(\rho\text{ro})\text{Metheus}$ program is contractually obligated to output.

The terms of contracts are determined by FairCovenant in accordance to all stake-holder demands. The core feature of Loare-Logic is to guarantee that Typed ρ -Calculus programs that cannot satisfy contractually-obligated standards of quality fail at compile time, *before* the program is run. This *static guarantee* of quality satisfies the dual mandate of:

- ensuring the safety and validity of model outputs;
- and saving money as full analytic runs and/or training runs are expensive.

4.2 Typed ρ -Calculus Expressions

At the top level, Typed ρ -Calculus formally specify the notion of atomic computational assets, and computation over said assets in the $(\rho\text{ro})\text{Metheus}$ universe. In particular, we define the following three type of asset composition:

- (*data-data composition*): arises when data scientists augment datasets by combining it with other sources of data. Here the pressure point is to ensure the dataset does not degrade in quality under augmentation.
- (*data-agent composition*): arises when an *agents* consume data either in training, or in inference.
- (*agent-agent composition*): arises when one *agent* calls on another *agent* to complete a sub-task. Now since the output of *agent* could also be interpreted as *data*, this composition may be decomposed as a sequence of *data-agent* compositions.

► **Remark 19.** We present the salient aspect of Typed ρ -Calculus while keeping the syntactical rigor to a minimum. This elevates the core features of the $(\rho\text{ro})\text{Metheus}$ computational environment, without inundating the readers with implementation details of a fully featured domain-specific programming language.

4.2.1 Elementary Typed ρ -Calculus Expressions

$$\begin{array}{ll}
 \text{atomic assets:} & \text{asset } a \ b := \text{data } a \mid \text{agnt } (a \rightarrow b) \\
 \text{elementary functions:} & \text{fns } := \text{train} \mid \text{infer} \\
 \text{covenant judgments:} & \kappa \in \mathcal{R}
 \end{array} \tag{8}$$

In the rest of this paper, we will use φ to refer to *atomic assets* and *elementary functions* in Typed ρ -Calculus, as well as inductively defined expressions built from said code. Some more commentary follows:

- (line 1): in Typed ρ -Calculus, the atomic **asset** are either datasets **data** or AI agents **agnts**. The expressions are parameterized by generic types a and b , which could be i.e. **String** or **Int** types. Note these parametrized types are not the same as the Covenant-Judgement, which specifies quality of data or model that has been audited by The (pro)Metheus Oracle.
- (line 2): the elementary functions are **train** which takes an **agnt** $(a \rightarrow b)$ and $(\text{data } a, \text{data } b)$ as input and outputs a trained **agnt** $(a \rightarrow b)$. Or **infer** which takes in **agnt** $(a \rightarrow b)$ as input along with query **data** a , and output inferred response **data** b .
- (line 3): the Covenant-Judgement of Typed ρ -Calculus is the entire covenant judgement universe as defined in def (14).

► **Example 20. (Document Data)** A set of raw conversation transcripts has type '**data String**'.

► **Example 21. (Dialogue Systems)** An automated dialogue system or chat-bot that accepts queries in string, and respond with raw text has type '**agnt (String $\rightarrow$ String)**'.

4.2.2 Group Composition over **asset** Expressions

Now given a set of words, we can "place a grammar" over this set, so that the words may compose into sentences. Similarly, if one conceptualizes **asset** as elementary programs or words, then one can "place an algebra" on **asset** so that they compose to form larger programs. The algebra of choice in Typed ρ -Calculus is the **Semigroup**.

► **Definition 22. (Semigroup)** A semigroup is an ordered pair $(\mathbb{S}, \times)$ such that $\mathbb{S}$ is a non-empty set, and $\times$ is an associative binary operation on $\mathbb{S}$.

► **Remark 23.** Addition and multiplication over integers are both examples of semigroups. In the case of integers, there is also an identity element associated with summation in 0. So that addition forms a *group* in $(\text{Int}, +, 0)$, and similarly for $(\text{Int}, \times, 1)$ in the case of multiplication. In the case of **asset**, the identity asset is not necessarily defined, hence the restriction to semigroup.

The next definition specifies **asset** composition by placing the Semigroup algebra on **data** :

$$\begin{array}{l}
 \text{Class Semigroup (data } a) \text{ where} \\
 \quad \diamond :: \text{data } a \times \text{data } a \rightarrow \text{data } a
 \end{array} \tag{9}$$

► **Example 24. (data Composition).** Given two block of raw texts s_1 and s_2 , each of type **data String**, we can concatenate the two blocks with $\diamond$ to form a new data set of type **data String**. Written:

$$s_1 \diamond s_2 :: \text{data String}.$$

Similarly, we place Semigroup operation on `agnt` as follows:

```
Class Semigroup (agnt (· → ·)) where
  >:: agnt (a → b) × agnt (b → c) → agnt (a → c)      (10)
```

► **Example 25. (agnt Composition).** Given an AI agent `agnt (String → String)` that consumes text and output text (i.e. a chat-bot), and another AI agent `agnt (String → Image)`' that consumes text and generates images. Then we can compose them with:

$$\left(\text{agnt (String} \rightarrow \text{String)} \succ \text{agnt (String} \rightarrow \text{Image)}' \right) :: \text{agnt (String} \rightarrow \text{Image)}.$$

And now we have a new AI agent that outputs images when prompted.

4.2.3 Elementary Functions over `asset` Expressions

Now we define two functions that operate on `asset`. These correspond to elementary operations in the (ρ) Metheus machine learning ecosystem.

```
train :: data a × data b × agnt (a → b) → agnt (a → b)      (11)
infer :: data a × agnt (a → b) → data b                      (12)
```

Commentary below:

- Function (11) takes in some dataset (`data a`, `data b`), and machine learning model `agnt (a → b)`, and `trains` the model to output a trained ML function `agnt (a → b)`. This signature is written for training discriminative models with labeled input pairs. In the case of generative models, `data b` is simply passed into the `train` function as the empty set.
- Function (12) takes a model `agnt (a → b)`, and accepts an input (or prompt in the case of LLMs) `data a`, `infers` its value and outputs a prediction or response of type `data b`.

This ends the discussion on Typed ρ -Calculus top level expression. In summary:

- The atomic `asset` of the (ρ) Metheus computational environment are `data` and `agnt`. They must be audited according to FairCovenant-defined rules before use client-side.
- The elementary machine learning functions that operate over said `asset` are `train` and `infer`, they take `data` and `agnt` as inputs, and output the appropriate `asset` in response.
- The composition operators `>` and `◇` compose `agnt` and `data` respectively. These semigroup functions, along with `train` and `infer`, allow the analyst to build complex Typed ρ -Calculus programs from atomic Typed ρ -Calculus `asset`.

In the next section, we introduce a novel concept termed Loare-Logic. It is the workhorse logical system used by the (ρ) Metheus compiler to validate Typed ρ -Calculus programs for Covenant-Compliance, before any AI-related computation is run.

4.3 Loare-Logic: Axiomatic Semantics with Legalistic-Hoare Logic

This section lays out the mathematical tools needed to enforce Covenant-Compliance of Typed ρ -Calculus expressions within the (pro)Metheus computational environment. Section 4.3.1 defines Loare-Logic syntactically as a set of Covenant-Judgement rules, as well the mechanisms of how pre and post-conditions are enforced to ensure Covenant-Compliance. Then we enumerate all the rules in the subsequent section:

- Section 4.3.2 assigns Covenant-Judgement onto each atomic `asset`. This is equivalent to designating legal status to each entity in (pro)Metheus, thereby reifying the type-covenant correspondence asserted in chapter 2.
- Sections 4.3.3 and 4.3.4 map `asset` composition onto Covenant-Judgement composition as defined by the product operation of definition (13). This ensures `asset` composed under $\diamond$ and $\rightarrow$ also witness the appropriate Covenant-Judgements, and are therefore Covenant-Compliant to the appropriate degree.
- Section 4.3.5 defines how Covenant-Judgement maps over the elementary function `infer` and `train`. That is to say: how does `inference` and `training` affect the output values' Covenant-Judgements.
- Section 4.3.6 provide some example proofs, demonstrating how Loare-Logic enforces Covenant-Compliance in complex Typed ρ -Calculus programs built inductively from `asset` primitives.

4.3.1 Loare-Logic Introduction

Loare-Logic is a formal system used by the (pro)Metheus compiler to analyze the quality of data and model assets used in Typed ρ -Calculus programs. Loare-Logic ensures the analytic output of said programs are covenant compliant according to FairCovenant-defined rules.

► **Definition 26.** (Loare-Logic) is a two-layered proof system that either generates a proof stating some Typed ρ -Calculus is covenant compliant, or terminates program execution for covenant violations. The system is layered as follows:

- A set of atomic and inductively defined rules assigning Covenant-Judgement to each expression in the Typed ρ -Calculus universe. Specifically, the following set of rules:

$$\text{rules} := \{\text{data-L}, \text{agent-L}, \text{compa-L}, \text{compb-L}, \text{seqL-L}, \text{seqr-L}, \text{seqd-L}, \text{train-L}, \text{infer-L}\},$$

assign each user-defined expression in Typed ρ -Calculus onto some covenant κ in the covenant universe $\mathfrak{R}$. In the case where the expression φ is some atomic `asset`, then the rule is written:

$$\text{rule: } \frac{}{\Gamma \models \varphi : \kappa}$$

If the expression is inductively defined using Semigroups operations $\diamond$ or $\rightarrow$, and/or using elementary functions such as `train` and `infer`, then the rule is written:

$$\text{rule: } \frac{\Gamma \models \varphi_1 : \kappa_1, \Gamma \models \varphi_2 : \kappa_2}{\Gamma \models \varphi_1 \varphi_2 : \kappa_3}$$

Stating that if expression φ_1 witness Covenant-Judgement κ_1 while φ_2 witness κ_2 , then their combined expression $\varphi_1 \varphi_2$ has Covenant-Judgement κ_3 . The exact rules to induce the judgement κ_3 is defined by (pro)Metheus compiler in conjunction with FairCovenant Foundation regulations.

- Building upon the rule primitives are a set of contractual obligations decorating Typed ρ -Calculus programs. They govern the execution of Typed ρ -Calculus programs, guaranteeing that the pre and postcondition of said programs are covenant compliant. Syntactically, this is written:

$$\{\{\Gamma \models x_1 : \kappa_1, \dots, x_n : \kappa_n\}\} \varphi(x_1, \dots, x_n) \{\{\Gamma \models y : \kappa' \geq \kappa''\}\},$$

where $y = \varphi(x_1, \dots, x_n)$, with $\kappa_1 \in \mathbf{K}_1, \dots, \kappa_n \in \mathbf{K}_n$, and $\kappa', \kappa'' \in \mathbf{K}'$.

Stating that:

- given some input assets $x_1, \dots, x_n$ of Covenant-Judgement $\kappa_1, \dots, \kappa_n$;
- and Typed ρ -Calculus expression φ that transforms the inputs onto output $y = \varphi(x_1, \dots, x_n)$;
- the output y must witness some Covenant-Judgement $\kappa' \geq \kappa''$.

The threshold value of data quality κ'' is called **the boundary condition on the output of computation** as enforced by Loare-Logic. It ensures the resultant prediction or response from the Typed ρ -Calculus code φ meets the minimum standards of quality as defined by the FairCovenant Foundation. The pre and postconditions defined in the brackets $\{\{\cdot\}\}$ will automatically "decorate" programs written by analysts in some specific instance of the (pro)Metheus computational environment.

Observe how Loare-Logic combines the syntax of type relations in example (15), and that of Hoare logic in definition (16) to express contractual enforcement of Typed ρ -Calculus programs w.r.t pre-agreed Covenant-Judgement. Now we specify the content of each *rule* enumerated in the set above, and provide examples of the Loare-Logic triple in action.

4.3.2 Elementary Covenant-Judgement over Computational Assets

The judgment of individual **data** and **agnt asset** are done by The (pro)Metheus Oracle according to FairCovenant Foundation defined rules. In this case we simply assume they are assigned some appropriately-defined Covenant-Judgement κ as follows:

$$\text{data-L} \quad \frac{}{\Gamma \models \text{data } a : \kappa} \qquad \text{agent-L} \quad \frac{}{\Gamma \models \text{agnt } (a \rightarrow b) : \kappa}$$

4.3.3 Semantics of **data** Composition w.r.t Covenant-Judgement

The following set of expressions takes the semigroup composition over **data** and **agnt** defined in expressions (9) and (10), and map them onto Covenant-Judgement composition as defined in expression (4).

$$\text{compa-L} \quad \frac{\Gamma \models \text{data } a : \kappa_1, \Gamma \models \text{data } a : \kappa_2, \kappa_1 \leq \kappa_2, \kappa_1, \kappa_2 \in \mathbf{K}}{\Gamma \models \text{data } a \diamond \text{data } a : \kappa_1}$$

$$\text{compb-L} \quad \frac{\Gamma \models \text{data } a : \kappa, \Gamma \models \text{data } b : \kappa', \kappa \in \mathbf{K}, \kappa' \in \mathbf{K}'}{\Gamma \models \text{data } a \diamond \text{data } b : \kappa \otimes \kappa'}$$

- In compa-L, we are given some **data** a of Covenant-Judgement κ_1 , and **data** a judged to be κ_2 , so that the first judgement κ_1 is less "valuable" than the second. Then in this case, the entire dataset default to the less valuable judgment κ_1 .

- In compb-L, there is some **data** a and **data** b , observe they are of different parameterized types. For example we may have **data** **String** and **data** **Image**, then their concatenation is the multi-modal dataset **data** **String** $\diamond$ **data** **Image**. In this case their covenants are drawn from different spaces, that is to say $\kappa \in \mathbf{K}$ and $\kappa' \in \mathbf{K}'$. Naturally the multi-modal dataset is in the product space $\kappa \otimes \kappa' \in \mathbf{K} \times \mathbf{K}'$.

► **Example 27.** (compa-L and compb-L) Suppose the following computational assets are loaded in the (pro)Metheus environment:

$$\Gamma \models \text{data Int} : \kappa_1, \quad \Gamma \models \text{data Int}' : \kappa_2, \quad \Gamma \models \text{data Image} : \kappa,$$

with $\kappa_1 < \kappa_2, \kappa_1, \kappa_2 \in \mathbf{K}, \kappa \in \mathbf{K}'$.

Then if the analyst build composite datasets, they will have the following inductively-defined Covenant-Judgement:

$$\Gamma \models \text{data Int} \diamond \text{data Int}' : \kappa_1, \quad \Gamma \models \text{data Int} \diamond \text{data Image} : \kappa_1 \otimes \kappa.$$

Note that since **data** **Int** and **data** **Int'** witness different elements of the same Covenant-Judgement space, the lower-valued Covenant-Judgement takes precedent. Where as **data** **Int** and **data** **Image** are drawn from different Covenant-Judgement spaces, thus the composite dataset rests in the product space of Covenant-Judgements or $\mathbf{K} \times \mathbf{K}'$.

► **Remark 28.** (Notation on type and Covenant-Judgement) In the code given in example 27, we use the notation $\Gamma \models \text{data } a : \kappa$ to signify a computational **asset** of type **data** a has Covenant-Judgement κ . Technically, it is more complete to write, i.e.:

$$\Gamma \models x :: \text{data } a : \kappa_1,$$

to signify that some dataset of value x of type **data** a has Covenant-Judgement κ_1 . However this introduces another symbol x , which is cumbersome. Hence in all of our examples we leave the variable name x implicit.

4.3.4 Semantics of **agnt** Composition w.r.t Covenant-Judgement

Now we consider how sequencing over **agnt** by $\rightarrow$ affect Covenant-Judgement.

$$\text{seq1-L} \frac{\Gamma \models \text{agnt } (a \rightarrow a) : \kappa_1, \Gamma \models \text{agnt } (a \rightarrow a)' : \kappa_2, \kappa_1 \leq \kappa_2, \kappa_1, \kappa_2 \in \mathbf{K}}{\Gamma \models \text{agnt } (a \rightarrow a)'' : \kappa_1}$$

$$\text{seqr-L} \frac{\Gamma \models \text{agnt } (a \rightarrow a) : \kappa_2, \Gamma \models \text{agnt } (a \rightarrow a)' : \kappa_1, \kappa_1 \leq \kappa_2, \kappa_1, \kappa_2 \in \mathbf{K}}{\Gamma \models \text{agnt } (a \rightarrow a)'' : \kappa_1}$$

$$\text{seqd-L} \frac{\Gamma \models \text{agnt } (a \rightarrow b) : \kappa_1, \Gamma \models \text{agnt } (b \rightarrow c) : \kappa_2, \kappa_1 \in \mathbf{K}, \kappa_2 \in \mathbf{K}'}{\Gamma \models \text{agnt } (a \rightarrow c) : \exists \kappa_3 \in \mathbf{K}'' \text{ s.t. } \kappa_3 = \text{proj}(\kappa_1 \otimes \kappa_2)}$$

$$\text{where } \text{proj}(\kappa_1 \otimes \kappa_2) := \text{arith}(\mathbf{K}'') \cdot \sqrt{\left(\frac{\text{arith}(\kappa_1)}{\text{arith}(\mathbf{K})}\right)^2 + \left(\frac{\text{arith}(\kappa_2)}{\text{arith}(\mathbf{K}')} \right)^2}.$$

- In seq1-L and seqr-L, one of the **agnt** witnesses a less valuable Covenant-Judgement than the other. In both cases the combined **agnt** $(a \rightarrow a)''$ defaults to the less valuable judgement κ_2 .

- In seqd-L, the **agnt** are parametrized by different types: **agnt** ($a \rightarrow b$) v.s. **agnt** ($b \rightarrow c$). Consequentially, their Covenant-Judgement are drawn from different covenant spaces, and the composed **agnt** ($a \rightarrow c$) is drawn from a third covenant space $\mathbf{K}''$. Thus, we have to search for the most suitable Covenant-Judgement $\kappa_3 \in \mathbf{K}''$ that we *expect* **agnt** ($a \rightarrow c$) to witness. We use the following procedure:
 1. We interpret the Covenant-Judgement **agnt** ($a \rightarrow b$) and **agnt** ($b \rightarrow c$) as two sides of a triangle, so that the Covenant-Judgement of **agnt** ($a \rightarrow c$) is the hypotenuse. The objective is now to find the length of this hypotenuse.
 2. Next we arithmetize κ_1 and κ_2 using the ‘arith’ function, and determine the length of the hypotenuse with the Pythagorean theorem.
 3. This arithmetized value of κ_3 is then un-arithmetized onto some $\kappa_3 \in \mathbf{K}''$.

► **Example 29. (seqd-L Composition).** Suppose we have this set of (pro)Metheus computational **asset** s, and their Covenant-Judgement:

$$\begin{aligned} \Gamma \models \text{agnt } (a \rightarrow b) : \kappa_2 \in \mathbf{K}, \quad \text{s.t. } \text{arith}(\kappa_2) &= 3, \quad \text{and } \text{arith}(\mathbf{K}) = 10, \\ \Gamma \models \text{agnt } (b \rightarrow c) : \kappa'_6 \in \mathbf{K}', \quad \text{s.t. } \text{arith}(\kappa'_6) &= 7, \quad \text{and } \text{arith}(\mathbf{K}') = 20, \\ \text{and } \text{arith}(\mathbf{K}'') &= 12, \quad \text{with } \mathbf{K}'' = \{\kappa''_0, \kappa''_1, \dots, \kappa''_{11}\}. \end{aligned}$$

Then we know that under $\rightarrow$ composition, the Covenant-Judgement of the combined value is:

$$\begin{aligned} \text{proj}(\kappa_2 \otimes \kappa'_6) &:= \text{arith}(\mathbf{K}'') \times \sqrt{\left(\frac{3}{10}\right)^2 + \left(\frac{7}{20}\right)^2} = 5.53, \\ \text{let } \text{agnt } (a \rightarrow c) &:= \text{agnt } (a \rightarrow b) \rightarrow \text{agnt } (b \rightarrow c), \\ \Gamma \models \text{agnt } (a \rightarrow c) &: \kappa''_4. \end{aligned}$$

Where going into the last line, we round 5.53 down to the value 5, which correspond to $\kappa''_4 \in \mathbf{K}''$.

4.3.5 Semantics of Function Application w.r.t Covenant-Judgement

The next rule defines how **train** changes Covenant-Judgement of the program output.

$$\begin{array}{c} \text{train-L} \\ \hline \frac{\Gamma \models \text{agnt } (a \rightarrow b) : \kappa_1, \Gamma \models \text{data } a : \kappa_2, \Gamma \models \text{data } b : \kappa_3, \kappa_1 \in \mathbf{K}, \kappa_2 \in \mathbf{K}', \kappa_3 \in \mathbf{K}''}{\Gamma \models \text{train}(\text{data } a, \text{data } b, \text{agnt } (a \rightarrow b)) : \exists \kappa_3 \in \mathbf{K} \text{ s.t. } \kappa_3 = \text{proj}(\kappa_1 \otimes \kappa_2 \otimes \kappa_3)} \\ \text{where } \text{proj}(\kappa_1 \otimes \kappa_2 \otimes \kappa_3) := \text{arith}^{-1} \left(\frac{\text{arith}(\kappa_1) \cdot \text{arith}(\kappa_2) \cdot \text{arith}(\kappa_3)}{\text{arith}(\mathbf{K}) \cdot \text{arith}(\mathbf{K}') \cdot \text{arith}(\mathbf{K}'')} \right). \end{array}$$

In train-L the **train** function **trains** some **agnt** ($a \rightarrow b$) using **data** a and **data** b , each one judged as κ_1 and κ_2 . In this case, the **trained** **agnt** ($a \rightarrow b$) is in the product space of the three Covenant-Judgements. The last rule defines how **infer** changes Covenant-Judgement of the program output.

$$\text{infer-L} \frac{\Gamma \models \text{agnt } (a \rightarrow b) : \kappa_1, \Gamma \models \text{data } a : \kappa_2, \kappa_1 \in \mathbf{K}, \kappa_2 \in \mathbf{K}'}{\Gamma \models \text{infer}(\text{data } a, \text{agnt } (a \rightarrow b)) : \exists \kappa_3 \in \mathbf{K}'' \text{ s.t. } \kappa_3 = \text{proj}(\kappa_1 \otimes \kappa_2)}$$

$$\text{where } \text{proj}(\kappa_1 \otimes \kappa_2) := \text{arith}^{-1} \left(\frac{\text{arith}(\kappa_1) \cdot \text{arith}(\kappa_2)}{\text{arith}(\mathbf{K}) \cdot \text{arith}(\mathbf{K}')} \right).$$

In `infer-L`, the `infer` function runs some `agnt` ($a \rightarrow b$) against input `data` a to output prediction or response `data` b . The resultant `data` b is in the product space.

► **Example 30. (train-L Composition)** Suppose we have this set of (pro)Metheus computational `asset` s, and their Covenant-Judgement:

$$\begin{aligned} \Gamma &\models \text{agnt } (a \rightarrow b) : \kappa \in \mathbf{K}, \text{ s.t. } \text{arith}(\kappa) = 5, \text{ and } \text{arith}(\mathbf{K}) = 10, \mathbf{K} = \{\kappa_0, \dots, \kappa_9\}. \\ \Gamma &\models \text{data } a : \kappa' \in \mathbf{K}', \text{ s.t. } \text{arith}(\kappa') = 25, \text{ and } \text{arith}(\mathbf{K}') = 30. \\ \Gamma &\models \text{data } b : \kappa'' \in \mathbf{K}'', \text{ s.t. } \text{arith}(\kappa'') = 18, \text{ and } \text{arith}(\mathbf{K}'') = 20. \end{aligned}$$

Then we know that under `training`, the Covenant-Judgement of the output `agnt` ($a \rightarrow b$)' is:

$$\begin{aligned} \text{proj}(\kappa \otimes \kappa' \otimes \kappa'') &:= \frac{5 \cdot 25 \cdot 18}{10 \cdot 30 \cdot 20} \cdot 10 = 3.75 \\ \text{let } \text{train}(\text{data } a, \text{data } b, \text{agnt } (a \rightarrow b)) &:= \text{agnt } (a \rightarrow b)' \\ \Gamma &\models \text{agnt } (a \rightarrow b)' : \kappa_2 \in \mathbf{K}'. \end{aligned}$$

Where going into the last line, we round down to the value 3, which correspond to $\kappa_2 \in \mathbf{K}$. Observe that training on lower quality data lowers the quality of the model `agnt` ($a \rightarrow b$).

► **Example 31. (infer-L Composition)** Suppose we have this set of (pro)Metheus computational `asset` s, and their Covenant-Judgement:

$$\begin{aligned} \Gamma &\models \text{agnt } (a \rightarrow b) : \kappa \in \mathbf{K}, \text{ s.t. } \text{arith}(\kappa) = 5, \text{ and } \text{arith}(\mathbf{K}) = 10, \\ \Gamma &\models \text{data } a : \kappa' \in \mathbf{K}', \text{ s.t. } \text{arith}(\kappa') = 19, \text{ and } \text{arith}(\mathbf{K}') = 30, \\ \mathbf{K}'' &= \{\kappa_0, \dots, \kappa_{24}\}. \end{aligned}$$

Then we know that under `inference`, the Covenant-Judgement of the output `data` b is:

$$\begin{aligned} \text{proj}(\kappa \otimes \kappa') &:= \frac{5 \cdot 19}{10 \cdot 30} \cdot 24 = 22.799 \\ \text{let } \text{infer}(\text{data } a, \text{agnt } (a \rightarrow b)) &:= \text{data } b \\ \Gamma &\models \text{data } b : \kappa_{21} \in \mathbf{K}''. \end{aligned}$$

Where going into the last line, we round 22.799 down to the value 22, which correspond to $\kappa_{21} \in \mathbf{K}''$.

► **Remark 32. (Computational efficiency of seqd-L, train-L, and infer-L Composition)** Observe that in the examples above, we used simple arithmetic to find the Covenant-Judgement of the composed value. Notably, in example 30 we did not run the `train` function, which would be quite expensive.¹¹ And in example 31, we did not run the `infer` function, whose cost is also not trivial. This satisfies the *static check* aspect of the (pro)Metheus environment feature, whereby code that could potentially lead to "bad" effects are not run at all. In this case the "bad" outcome may include:

- inference on poor input data, leading to low quality output;

¹¹ Indeed a single training run may cost 10s of millions of USD.

- training on poor quality data, leading to lower quality model `agnt`.

► Remark 33. (**The role of FairCovenant Foundation in defining inductive** Loare-Logic judgements) Observe that the `proj()` function we defined for `train` can never improve the Covenant-Judgement of the trained `agnt`, it can only degrade its value. This `proj()` may be too stringent in production. In reality the stringency of `proj()` is context dependent, and will be defined in conjunction with stakeholders as work streams within the FairCovenant Foundation.

4.3.6 Proof of Covenant-Compliance with Loare-Logic

This section places the elementary concepts of Loare-Logic introduced in the previous section in the context of an example, and walk through how a (ρ)Metheus compiler uses Loare-Logic to enforce Covenant-Judgement compliance.

► **Example 34. (Proof of Covenant-Compliance)** Given Covenant-Judgements sets:

$$\begin{aligned} \text{data Covenant-Judgement : } \mathbf{K}_a &= \{\kappa_1^a, \dots, \kappa_{10}^a\}, \quad \mathbf{K}_b = \{\kappa_1^b, \dots, \kappa_{10}^b\}, \\ \mathbf{K}_c &= \{\kappa_1^c, \dots, \kappa_{10}^c\}, \quad \mathbf{K}_d = \{\kappa_1^d, \dots, \kappa_{10}^d\}, \\ \text{function Covenant-Judgement : } \mathbf{K}_{f_1} &= \{\kappa_1^{f_1}, \kappa_2^{f_1}, \kappa_3^{f_1}, \kappa_4^{f_1}, \kappa_5^{f_1}\}, \quad \mathbf{K}_{f_2} = \{\kappa_1^{f_2}, \kappa_2^{f_2}, \kappa_3^{f_2}\}, \\ \mathbf{K}_{f_3} &= \{\kappa_1^{f_3}, \kappa_2^{f_3}, \kappa_3^{f_3}\}. \end{aligned}$$

And available computational `asset` with Covenant-Judgement:

$$\begin{aligned} \Gamma \models x_1 &:: \text{data } a : \kappa_9^a, \quad \Gamma \models x_2 :: \text{data } a : \kappa_6^a, \quad \Gamma \models x_3 :: \text{data } a : \kappa_7^a, \\ \Gamma \models y &:: \text{data } b : \kappa_8^b \\ \Gamma \models \mathbf{fn}_1 &:: \text{agnt}(\text{data } a \rightarrow \text{data } c) : \kappa_5^{f_1}, \\ \Gamma \models \mathbf{fn}_2 &:: \text{agnt}(\text{data } c \rightarrow \text{data } d) : \kappa_2^{f_2}. \end{aligned} \tag{13}$$

Additionally, we know that there is some:

- $\kappa_j^{f_3} \in \mathbf{K}_{f_3}$ s.t. $\Gamma \models \text{agnt}(\text{data } a \rightarrow \text{data } d) : \kappa_j^{f_3}$.
- $\kappa_i^d \in \mathbf{K}_d$ s.t. $\Gamma \models \text{asset} : \kappa_i^d$.

Now suppose the user builds a Typed ρ -Calculus expression as follows:

$$d_* = \text{infer } x_1 \left(\underbrace{\text{train} \left(\underbrace{x_2 \diamond x_3, y}_{(2)}, \underbrace{(\mathbf{fn}_1 \rightarrow \mathbf{fn}_2)}_{(1)} \right)}_{(3)} \right). \tag{14}$$

(4)

Prompting the program to train some model $\mathbf{fn}_1 \rightarrow \mathbf{fn}_2$ on dataset $(x_2 \diamond x_3, y)$, before doing inference on input x_1 . Now given a client-defined set of boundary condition on the quality of the output to expression (14) with:

$$\kappa_*^d \geq \kappa_5^d.$$

Stating the output of expression (14) must witness some covenant-value greater than or equal to κ_5^d . Then we can construct a proof that witnesses the Covenant-Compliance of expression (14) as follows:

1. Using the rule `seqd-L`, function composition under $\rightarrow$ reduces to:

$$\text{seqd-L} \frac{\Gamma \models \mathbf{fn}_1 : \kappa_5^{f_1}, \quad \Gamma \models \mathbf{fn}_2 : \kappa_2^{f_2}}{\Gamma \models (\mathbf{fn}_1 \rightarrow \mathbf{fn}_2) : \kappa_3^{f_3}}$$

Where we determine the value of $\kappa_3^{f_3}$ with:

$$\text{proj}(\kappa_5^{f_1} \otimes \kappa_2^{f_2}) = 3 \times \sqrt{\left(\frac{5}{5}\right)^2 + \left(\frac{2}{3}\right)^2} = 3.60 \sim 3.$$

So we have in the conclusion: $\Gamma \models (\mathbf{fn}_1 \rightarrow \mathbf{fn}_2) : \kappa_3^{f_3}$.

2. Using the rule `compa-L`, we have:

$$\text{compa-L} \frac{\Gamma \models x_2 : \kappa_6^a, \quad \Gamma \models x_3 : \kappa_7^a}{\Gamma \models (x_2 \diamond x_3) : \kappa_6^a}$$

3. Using the rule `train-L`, we have:

$$\text{train-L} \frac{\Gamma \models (x_2 \diamond x_3) : \kappa_6^a, \quad \Gamma \models y : \kappa_8^b, \quad \Gamma \models (\mathbf{fn}_1 \rightarrow \mathbf{fn}_2) : \kappa_3^{f_3}}{\Gamma \models \text{train}(x_2 \diamond x_3, y)(\mathbf{fn}_1 \rightarrow \mathbf{fn}_2) : \kappa_2^{f_3}}$$

Where we determined the value of projected Covenant-Judgement with:

$$\text{proj}(\kappa_6^a \otimes \kappa_8^b \otimes \kappa_3^{f_3}) = 3 \times \frac{6 \times 8 \times 3.60}{10 \times 10 \times 3} = 1.72 \sim 2.$$

4. Finally using `infer-L` we have:

$$\text{infer-L} \frac{\Gamma \models x_1 : \kappa_9^a, \quad \Gamma \models (\mathbf{fn}_1 \rightarrow \mathbf{fn}_2) : \kappa_2^{f_3}}{\Gamma \models d_* : \kappa_6^d}$$

Where we inductively define the output Covenant-Judgement with:

$$\text{proj}(\kappa_9^a \otimes \kappa_2^{f_3}) = 10 \times \frac{9 \times 2}{10 \times 3} = 6.$$

And so we have:

$$\Gamma \models d_* : \kappa_6^d,$$

witnessing the expected output value of expression (14) to be greater than the boundary condition of κ_5^d . Thus the (pro)Metheus has used Loare-Logic to generate a proof stating the computation is Covenant-Compliant

We can rewrite this entire proof in shorter format with:

$$\frac{\frac{\Gamma \models \mathbf{fn}_1 : \kappa_5^{f_1} \quad \Gamma \models \mathbf{fn}_2 : \kappa_2^{f_2}}{\Gamma \models (\mathbf{fn}_1 \rightarrow \mathbf{fn}_2) : \kappa_3^{f_3}} \quad \frac{\Gamma \models x_2 : \kappa_6^a \quad \Gamma \models x_3 : \kappa_7^a}{\Gamma \models (x_2 \diamond x_3) : \kappa_6^a} \quad \Gamma \models y : \kappa_8^b}{\frac{\Gamma \models \text{train}(x_2 \diamond x_3, y)(\mathbf{fn}_1 \rightarrow \mathbf{fn}_2) : \kappa_2^{f_3}}{\Gamma \models d_* = (\text{infer } x_1(\mathbf{fn}_1 \rightarrow \mathbf{fn}_2)) : \kappa_6^d}} \quad \Gamma \models x_1 : \kappa_9^a$$

This is known as a proof tree, or in this context: a Loare-Logic proof tree.

This exact proof is generated by the (ρ) Metheus compiler before any training or inference is conducted. So that if the proposed computation in expression (14) is Covenant-Compliant, then this proof can be produced as a witness. Otherwise the compiler halts the computation and asks the analyst to input higher quality **asset** into the Typed ρ -Calculus expression, so as to satisfy client-defined notions of quality. This is called static guarantee of Covenant-Compliance. The Typed ρ -Calculus language thus has satisfies the following desiderata:

- *(Monetary)* save money on training and inference by checking the *expected* quality of the final computation, *before* any expensive computation is done client side.
- *(Legality)*: Loare-Logic has shifted the burden of proof from the analyst to the compiler. Thereby opening up a greater set of inference tasks that can be done by analysts in high-stake settings with legal responsibilities.
- *(Composability)*: the proof tree generated by Loare-Logic used elementary definitions of Covenant-Judgement generated by The FairCovenant Foundation, and boundary conditions of inference quality defined by the client. It then inferred the final Covenant-Judgement of the computation without further human input, thereby greatly augmenting the capacity of regulators to audit the computation trees written by analysts deep in the bowels of their work routine.
- *(End-to-end-oversight)*: although the computation in expression (14) appear simple and "one lined," it in fact may span many days or even weeks. As the **train** function alone may be an industrial training run of some deep learning model. The (ρ) Metheus compiler could audit this industrial routine a-priori in its entirety, or audit aspects of the computation over the course of its realization. This gives flexibility as there may be some discrepancy between the *expected* quality of the output under Covenant-Judgement, and its actual quality after a training run. This flexibility is particularly salient in the case of deep learning models, which feature nonconvex objective functions with indeterminate quality a-priori training.
- *(Flexibility)*: in this example we used author-defined notions of `proj()` and arithmetization functions. However in real life, it would be the stakeholders that define said functions. They may dial up or down the restrictiveness of the proof at each step along the proof tree generated above. Thereby controlling the amount of computation they deem acceptable.

► **Example 35.** (Typed ρ -Calculus **program decorated by Loare-Logic triples**) We can rewrite example (34) by decorating the expression with Loare-Logic triples with:

$$\begin{aligned}
 & \left\{ \left\{ \Gamma \models x_1 : \kappa_9^a, x_2 : \kappa_6^a, x_3 : \kappa_7^a, y : \kappa_8^b, \mathbf{fn}_1 : \kappa_5^{f_1}, \mathbf{fn}_2 : \kappa_2^{f_2} \right\} \right\} \\
 d_* = & \text{infer} \\
 & x_1 \\
 & \text{train} (x_2 \diamond x_3, y) \ (\mathbf{fn}_1 \rightsquigarrow \mathbf{fn}_2) \\
 & \left\{ \left\{ \Gamma \models \kappa_*^d \geq \kappa_5^d \right\} \right\}
 \end{aligned} \tag{15}$$

This conforms to the syntax presented in definition (26).

5 Extending (pro)Metheus with Causal-Intelligence

This section accomplishes four items:

- Extends the basic Typed ρ -Calculus language to incorporate Pearl's do-Calculus for causal discovery and counterfactual queries.
- Enrich the definition of Loare-Logic so that its Covenant-Judgements cover the language extension stated in point one.
- Demonstrate some practical examples of how Typed ρ -Calculus enriched with causal discovery logic can be used to diagnose the causality of events in practice. And how Loare-Logic is threaded through this analysis to guarantee the outcome of causal inference contractually meets the standards set forth by the client.

5.1 Why Estimating Causality is Valuable in Applied AI

5.1.1 A Brief History of AI

The march towards general artificial intelligence has been a long one. In "the beginning," scholars built AI using hard-coded logical rules that require an immense amount of effort, but cannot generalize outside of a narrow predefined domain. Two prominent examples are the dialogue system of ELIZA in the 1960s, and early chess programs that used tree search to compute the best next move given the current board. In both cases, the combinatorial explosion of possibilities proved to be a *hard ceiling* for classical methods. The chatbot ELIZA could not converse beyond a few sentences. And chess programs cannot scale to solve Go: a far more subtle game featuring 10^{170} positions, versus chess' 10^{50} possible moves.

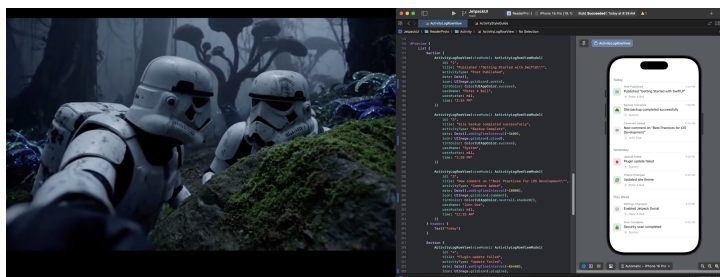


■ **Figure 9** ELIZA was an early chatbot developed at MIT in the late 1960s. It used hard coded grammar and semantics rules to fool the human eye, however it was quite brittle in real life. The developers thought they could improve the system to pass the Turing test within the summer, but this proved to be too ambitious given the techniques and hardware of the era. Decades later, this outcome was finally achieved with large language models (LLMs). And yet the work is far from finished. From a technical standpoint, LLMs are information retrieval systems with high recall and dubious precision. From an allegorical standpoint, LLMs are "ghosts of the internet" echoing endlessly to possess the human interlocutor. They cannot think anymore than ghosts can die (again).

What was needed was not more rules, but also a more principled way to express uncertainty about knowledge of the underlying reality itself. This underlying reality could manifest itself in i.e. the possible next best move on some game board, or the ambiguous meaning of words in a sentence. One principled way of reasoning about uncertainty is expressed with Bayesian inference over graphical models, whose structure represents intuitive or subjective notions of uncertainty of the underlying reality. The advent of internet-scale datasets and GPUs pushed Bayesian-network-based methods into

industry, culminating in the deep learning revolution of the 2010s. The Bayesian perspective proved flexible enough to express classical statistical workhorses such as regression. And the combination of these disciplines led to several major breakthroughs:

- Breakthroughs such as convolutional neural networks (CNNs), which could identify objects in images and video with higher rate of accuracy than humans. Thereby laying the ground work for truly automated drones and self-driving cars, which could now navigate in human-environments using cameras as an elementary sensor.
- Breakthroughs such as AlphaGo, which defeated the world's top-ranked human players. This feat was previously unthinkable with classical methods that relied on brute force or even heuristically-pruned tree search. However, with a combination of CNNs that represent the Go-board in compressed format, and using deep reinforcement learning to define the best strategy to do tree search, the game of Go has finally been solved by a computer in a definitive manner.
- Breakthroughs such as large language models, which could converse in "general" terms with a human interlocutor. Thus effectively defeating the Turing test, previously the "most hallowed bar" of human intelligence. Moreover, the methods underlying LLMs proved robust enough to handle a variety of time-series data, including multi-modal data (read: music and video) . This makes it possible to generate digital avatars, music, not to mention computer code itself. This is profound: LLMs are automating a significant chunk of the artisanal market for human talent. Thus AI has penetrated into the vocational class of artists, musicians, and junior software programmers.



■ **Figure 10** Multi-modal LLMs are penetrating the artisanal class of jobs. (Left): The Instagram account **StormtrooperVlogs** produce entertaining Star Wars videos with high production value. All for a fraction of the cost: sans green screen, special effects capex, actors who cannot act, or George-Lucas-flavored dialogue. (Right): vibe coding has become both an internet meme and very real productivity driver. Entry level coders used to write the same boiler plate code over and over again. Now they are automated away with LLM-agents that sample from GitHub's vast repo of existing code.

5.1.2 The Next Frontier of Applied AI is Causal-Intelligence

The next frontier in artificial intelligence is to penetrate the professional caste of white-collar workers. In this domain, LLMs may suffice at the entry level, whereby most of the work is document retrieval and summarization. That is to say: grunt work.¹² However at the associate level and above, AI agents must develop a refined understanding of how and why the world works if it is to be truly productive.¹³ And yet it is at this exact level that current deep learning methods shall hit a ceiling, as hard and definite as the ceiling that plagued the classical AI methods. This glass-ceiling in intelligence is *the*

¹² Cue "plz fix, k thnx" joke.

¹³ One has to wonder: do electric sheep dream of office politics?

barrier of causality. Unlike prior walls in deep learning over the last decade, this wall is baked into the syntax of the mathematical language underpinning deep learning itself. Therefore this ceiling cannot be shattered by bigger data or more compute.

► **Example 36. (Limitations of LLMs in Medical Reasoning)** [S25] In the paper "Fidelity of Medical Reasoning in Large Language Models," Bedi et.al. investigates "do [LLMs] reason through medical problems or exploit statistical patterns in their training data?" The result is large negative, they found that: "A system dropping from 80% to 42% accuracy when confronted with a pattern disruption would be unreliable in clinical settings, where novel presentations are common. The results suggest that these systems are more brittle than their benchmark scores suggest."

Table. Model Performance on Original and None of the Other Answers (NOTA)- Modified Questions*

Model	Accuracy, % (No./total No.)		Accuracy drop, % (No./total No.) [95 % CI]
	Original	NOTA-modified	
1	92.65 (63/68)	83.82 (57/68)	8.82 (6/68) [2.70-18.92]
2	95.59 (65/68)	79.41 (54/68)	16.18 (11/68) [10.81-29.73]
3	88.24 (60/68)	61.76 (42/68)	26.47 (18/68) [17.57-39.19]
4	92.65 (63/68)	58.82 (40/68)	33.82 (23/68) [24.32-47.30]
5	85.29 (58/68)	48.53 (33/68)	36.76 (25/68) [28.38-51.35]
6	80.88 (55/68)	42.65 (29/68)	38.24 (26/68) [27.03-51.35]

* This table compares performance on 68 clinician-validated questions. Original accuracy refers to performance on questions in their standard format, while NOTA-modified accuracy shows performance when the correct answer was replaced with "None of the other answers" (NOTA). Models are ordered by increasing accuracy drop. CIs were calculated using the McNemar test for paired nominal data.

This brittleness arises from the fact that LLMs can only capture statistical correlation, and cannot generalize out of sample. The fragileness is structural to the language of statistics itself, this can be remedied by a new set of tools that explicitly model causality.

Deep-learning-based AI models such as LLMs will inevitably hit the ceiling of causality. This ceiling cannot be shattered by bigger data or more compute.

The limitation of deep learning is not lost to those who study causality:



Judea Pearl
@yudapearl



Here is the paragraph in BOW where I make use of Plato Cave analogy:
One aspect of deep learning does interest me: the theoretical limitations of these systems, primarily limitations that stem from their inability to go beyond rung one of the Ladder of Causation. This limitation does not hinder the performance of AlphaGo in the narrow world of go games, since the board description together with the rules of the game constitutes an adequate causal model of the go-world. Yet it hinders learning systems that operate in environments governed by rich webs of causal forces, while having access merely to surface manifestations of those forces. Medicine, economics, education, climatology, and social affairs are typical examples of such environments. Like the prisoners in Plato's famous cave, deep-learning systems explore the shadows on the cave wall and learn to accurately predict their movements. They lack the understanding that the observed shadows are mere projections of three-dimensional objects moving in a three-dimensional space. Strong AI requires this understanding.

- Despite its success, the system is fundamentally limited in that the statistical language underpinning said models can only express correlation, not causation. Pearl was an early proponent of applying Bayesian inference over probabilistic graphical models to express correlation in data. Although he was awarded the Turing prize for this work, Pearl nonetheless developed the field of causality to address the limitation of Bayesian inference that he himself pioneered.¹⁴
- Presently, the elementary concepts all machine learning scientists must master are Bayesian networks, optimization techniques expressed using linear algebra, and various formulations of regression. However neither advanced master students nor PhD's in machine learning are exposed to the questions of causality; in fact they are often actively discouraged from looking in this direction. Consequentially, the current field of AI is somewhat blind to the topic at the human level, especially in light of the LLM craze.
- In more blunt terms, the contribution of Causal-Intelligence is *undervalued* by a factor of ten, if not more. Here the magnitude is important, as the median compensation at OpenAI among machine learning scientists is 1.2 million USD, while Facebook is paying some engineers 100 million USD over four years to develop native video generation pipelines.
- However no amount of gratuitous capex will help said models find causality. This does not affect Facebook, whose use case for generative AI is in the realm of the mundane. However in critical decision making roles where (pro)Metheus occupies, causal inference is a must-have. Over time, the market will rationalize and reappraise the value of Causal-Intelligence, and Pearl's work on causation will surely be added to any core curriculum in machine learning.
- Determining causality is particularly relevant w.r.t (pro)Metheus' target market. For example, in the context of insurance, proving beyond a reasonable doubt that some party caused an event to occur is the first step in assigning responsibility to this party, and soon thereafter: liability. Thus there is a direct line from causality to the exchange of money that does not exist in more primitive deep learning models. The monetary case of Causal-Intelligence could not be more clear:

Causality = Responsibility = Liability

Once machine intelligence can understand causality, the industry can finally move beyond artificial intelligence inference to *machine intelligence adjudication*.

5.2 Technical Preliminary

This section introduces the basic concepts needed to speak of causality in a formal manner. Unlike prior chapters whose intellectual basis is drawn from programming language design and classical logic, the content of this section is largely drawn from Pearl's do-Calculus and those who extend it. Namely [Bar25], [J P17], and [Pea09]. We take a narrow pedagogical pathway through the subject so as to present a minimal slice, with an emphasis on defining the concepts needed to specify causality. Knowledge of probability, graphical models, and reinforcement learning is assumed. Readers who are unfamiliar with these concepts, or wish to see how we use notation to express said concepts, should read section 6. Finally, any metaphysical discussion on the nature of causality itself, while entertaining, is simply ignored.

¹⁴ The Turing award and the Gödel Prize are the highest honor in computer science, comparable to the Nobel prize.

5.2.1 The Forward and Backward Question of Causality

All causal questions posed against (pro)Metheus can be divided into two categories:

- **The Forward Question:** in this direction, the system is given a proposition of causality, and must construct a proof from the available dataset witnessing this proposition. Additionally, the system may be asked to present a confidence estimation along with the proof given.
- **The Backward Question:** in the reverse direction, the system is given a dataset with embedded causal information, and must determine one or several causal propositions evidenced in data. Moreover, the system may be asked to rank the proposition by how well they fit the evidence.

Observe: the computational complexity or demand of the two questions are categorically different. The forward question may be determined, while the backward question is often times ill-posed. While the forward question is solved by a simple application of scoring datasets against a proposed causal graph, the backward question may be parsimoniously solved by causal reinforcement learning.

► **Remark 37. (Hierarchy of Proofs)** Observe that the proposition and proofs in this case are different from the Loare-Logic proof trees of section 4.3.6. Where as Loare-Logic generates type-level proofs witnessing the validity or Covenant-Compliance of the computation in "cyberspace," the Causal-Intelligence proof is a term-level proof demonstrating the likely causality of some event in "real space." In short, the Loare-Logic-proof is a judgement on the process of arriving at a conclusion, while the Causal-Intelligence proof is a judgement of the underlying reality. This is what we mean by "two-tiered proof system" in the abstract.

The rest of this chapter will introduce the relevant aspect of do-Calculus used to infer causality from data. This is followed by causal reinforcement learning (CRL), which for our purpose is a "strategic" way to determine the most likely causal graph. Then we will express CRL with Typed ρ -Calculus, so as to augment (pro)Metheus with the capacity to understand causality.

5.2.2 The Ladder of Causation

This section contextualizes classical machine learning techniques w.r.t hierarchy of causal concepts.

Layer	Activity	Machine Learning
Association $p(y x)$	Seeing: what does an observation say about the underlying state?	(Un)supervised learning
Intervention $p(y do(x))$	Doing: how would an action lead to a different outcome.	Reinforcement learning
Counterfactual $p(y_x y', x')$	Imagining: why? What if the agent acted differently.	Explanation/Transparency

■ **Table 3** Pearl's causal ladder and their colloquial analogues.

The causal hierarchy of table 3 tiers the sophistication of questions queried on data into three levels:

1. (*Association*): invoking purely statistical relationships defined by raw data. For instance, observing a customer who buys toothpaste makes it more likely that they buy floss. Such association can be inferred directly from the observed data using conditional expectation.
2. (*Intervention*): The second level ranks higher than association because it involves not just seeing what is, but changing what is seen. One question at this level is: "what happens if we double the price of toothpaste?" Such questions cannot be answered from sales data alone, because they

involve a change in customers behavior in response to the new price. Customer choices under the new price structure may differ substantially from the past.

3. (*Counterfactual*): A typical question in this category is “what if I were to act differently?” Thus necessitating retrospective reasoning. Counterfactual models are placed at the top of the hierarchy because they subsume intervention and associational questions. If we have a model that can answer counterfactual queries, we can also answer questions about interventions and observations. However, the reverse is not true. Associational questions cannot be used to answer intervention or counterfactual questions. Therefore this ladder is a hierarchy delineating the explanatory power of statistical models [25d; Pea09].

The aim of standard statistical analysis (i.e. regression and estimation) is to assess parameters of a distribution from samples drawn from said distribution. Then using said parameters, one infers associations among variables, and update parameters given new data. One underlying assumption here is that data is drawn i.i.d, which proves restrictive in reality and severely limit the capacity of many statistical models to generalize out of domain. Causal analysis not only infers probabilities under static conditions, but also the dynamics of beliefs under changing conditions. These changes are induced by treatments, new policies, or external interventions [Pea10]. The various levels of the causation ladder is outlined in table 3.

Furthermore, the relationship between associational/correlational and causal concepts with statistical methods is as follows:

- **An associational concept** is a relationship that can be defined in terms of a joint distribution of observed variables. Examples include correlation, regression, dependence, conditional independence, likelihood, odds ratio, marginalization, conditionalization, etc. All statistical models utilizing deep learning tools fall under this category. Language models in particular are a kind of auto-regression on time-series data.
- **A causal concept** is any relationship that cannot be defined from the distribution alone. Examples of causal concepts are randomization, influence, effect, confounding, “holding constant,” structural coefficients, faithfulness/stability, instrumental variables, intervention, explanation, and attribution.

Associational concepts are readily expressed by the language of probability. However, probabilistic models, no matter how complex, cannot express simple concepts such as “wet pavement does not cause rain.” In theory, causal relations can only be determined with randomized controlled experiments. However such experiments may be expensive to run, or its collected data may be gated under privacy considerations. Sometimes it is simply unethical to run such experiments altogether, i.e.: does inhaling vape cause cancer in children 10 and under?

In order to express causation, the language of probability must be augmented with new symbols or algebra to express causal concepts that are intuitive to the human mind. This is the language of do-Calculus applied to the transformation of directed graphical models, it allows (pro)Metheus to estimate causal relations from observational data alone. It does so by translating basic intervention or counterfactual queries into operations on graphical models, questions such as does treatment x cause outcome y . Finally it uses observational data alone to accept or reject said queries.

While large language models replace analysts, (pro)Metheus Causal-Intelligence replaces associates.

This ends the prerequisite preliminary needed to set up the (pro)Metheus Causal-AI adjudication environment. The next subsection augments (pro)Metheus with causal intelligence. Those who

are unfamiliar with do-Calculus and causal reinforcement learning may read section 6 for further technical preliminaries.

5.3 Case Study: Transnational Supply Chain Insurance

This section proceeds in this order:

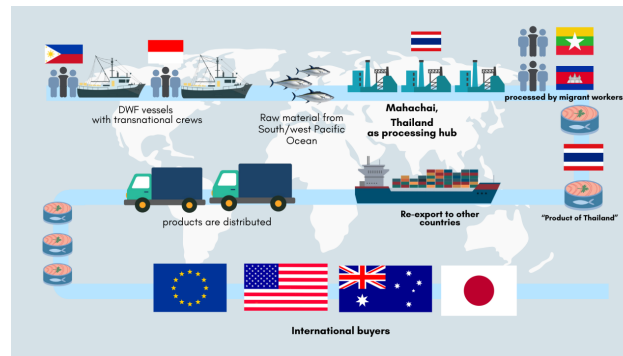
1. First, it presents one use case for (pro)Metheus in the context of transnational trade.
2. Then it abstracts the use cases into a general system diagram and workflow.
3. Next, it poses the system flow as an off-policy causal reinforcement learning problem.
4. Finally, key aspects of the workflow is expressed directly in the syntax of Typed ρ -Calculus itself, so that the (pro)Metheus compiler can generate static proofs confirming the validity of the adjudication process itself.

This section expands upon the basic example given in section 1.1.2, and presents more detailed use cases of how the (pro)Metheus Oracle determines causality given data from multiple sources.

5.3.1 User Journey

In the transnational supply chain setting, parcels carrying perishable goods move from a commodity center in East Africa (E.A.C.C.) to a special economic development zone in East Asia (SEZ).

Lifecycle of a Parcel and Stakeholder Touchpoints



■ **Figure 11** A transnational supply chain spans many jurisdictions, featuring heavy use of paper work, and variable degree of tech penetration. Every stakeholder along the supply chain contribute to the final quality of the parcel, and emits data during the process that witness their liability or innocence in the event of a faulty delivery.

1. The *seller* from E.A.C.C. is situated in an underdeveloped economic zone. Here, documentation for quality of parcel is sparse, and technology penetration is low. The seller grows perishable commodity locally, and sells them to the E.A. *reseller*. The reseller marks up the parcel by 50% and ships it across the ocean, with a transit time of 40 days.
2. The E.A. *reseller* deposits the parcel to the *local reseller* in the SEZ, who marks it up an additional 75% and warehouses the product for 14 days.
3. The local reseller then delivers the parcel to *point of sale* (P.O.S) in the SEZ. The P.O.S sells it at retail price with 10% markup.
4. The *buyer* buys parcel and takes it home. He opens the product and finds that it is *spoiled*.

Pertinent Points under Investigation

The questions here are:

- Who is responsible for the final condition of the product?
- How much should the responsible parties pay?
- Whomst are they paying? How are the funds disbursed?
- Which players along the supply chain should be investigated further? If the players have reputation rating, how much should their ratings be impacted due to failure to maintain parcel quality?

Data Accessible to the (ρro) Metheus Causal-Intelligence Oracle

This is a fundamentally a risk management problem *a-priori* parcel purchase, and claims adjudication process *a-posteriori* parcel delivery. In order to prosecute the outcome appropriately, (ρro) Metheus Causal-Intelligence collects data *a-priori* via the following user touchpoints:

- Both the seller and reseller in E.A.C.C will have a mobile app equipped with a camera, he takes a picture of the package before dropping it off with the reseller. The image is GPS and timestamped. This is a trivial technical requirement as mobile phone penetration in East Africa is 100%.
- The local resellers at the S.E.Z. shares data from their warehouse over the 14 day period, this may include:
 - continuous measurements of warehouse temperature,
 - as well as timestamp and duration of when products are loaded, etc.
- (ρro) Metheus will also have access to temperature gauge within the shipping container as it completes its transit across the ocean.
- Finally, (ρro) Metheus will have access to a stream of general world events that is ancillary to the parcel transit process. This may include weather reports, news reports of exogenous shock, etc.

In summary, the parcel lifecycle is a data-generation process. The (ρro) Metheus Causal-Intelligence Oracle will work with the potentially liable parties along the entire supply chain to assign liability based on this data stream.

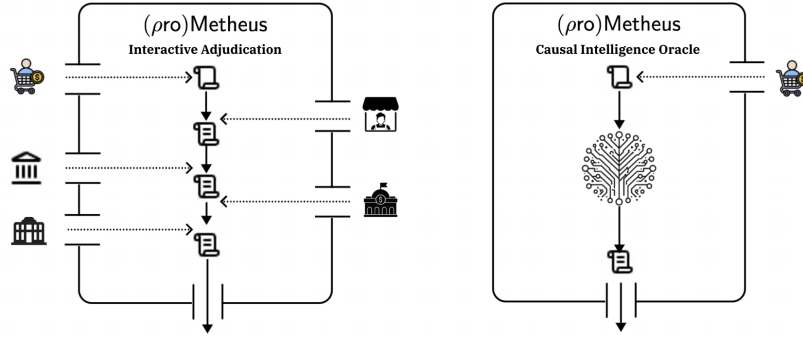
5.3.2 The (ρro) Metheus Adjudication Game

There are two possible adjudication regimes: the interactive and the Causal-Intelligence automated process. Referencing fig (12):

- **Policyholder Mutual Accountability Interactive Application:** in this arrangement, each stakeholder or policyholder along the supply chain interacts with a natural-language-based dialogue-based system. They proceed in a two stages:
 1. **(Defend):** in stage 1, each party may provide additional data proving the condition of the product before and after their touch point, so as to defend their innocence.
 2. **(Accuse):** in stage 2, the party may provide evidence, or plausible explanation - stating that another member(s) along the same supply chain is liable.

Finally, a certified examiner will perform a final review of the information and decide on an outcome. Payouts will be allocated according to defined policies, and the associated parties' insurance premium will be adjusted appropriately. The interactive setting is also called the *expert demonstration* loop, and it accomplishes four objectives:

1. (*Operational leverage*): is gained in the adjudication process, as policyholders are partaking in the investigation process.
 2. (*Mutual accountability*): is enculturated, where various parties along the supply chain hold each other accountable for the quality of the parcel.
 3. (*Community policy retention*): each member along the supply chain are part of the same co-op, and have shared responsibility to deliver the product unmarred. In the event where "bad" events occur, their voice is heard as they are given a chance to prove their innocence.
 4. (*Training data*): the interactive adjudication process is also a data labeling process. It will be used to train the (pro)Metheus Causal-Intelligence Oracles, enabling automation.
- **The (pro)Metheus Causal-Intelligence Automated Oracle:** in this setting, the (pro)Metheus Oracle validates the parcel history, and decides on an outcome based on its internal Causal-Intelligence engine. This process is legally compliant as the (pro)Metheus program judging the claim is expressed in Typed ρ -Calculus, so that each step of its computation graph observes the necessary legal regulations.



■ **Figure 12** (*Left*): the (pro)Metheus interactive adjudication application is a *mutual accountability game*, whereby each policy holder is asked to defend their role in the supply chain by submitting additional data proving their innocence, and then accuse a neighbor in the chain of malpractice that degrades the final quality of the parcel. Finally a certified examiner passes judgement and closes the case with appropriate payouts and adjustments. (*Right*): the (pro)Metheus Causal-Intelligence automated adjudication Oracle ingests the claim along with data generated by the underlying commodity flow, and decide on the outcome directly sans mandatory human input.

The (pro)Metheus Causal-Intelligence Oracle grants immense operational leverage and risk assessment abilities. Thus allowing us to **insure the uninsurable**.

5.3.3 Posing the Causal-Intelligence Adjudication Problem

The two user journeys of section 5.3.2 differ in degree of human interaction, but have the same goal of identifying the party at fault, and pricing the blame. From an ML-driven automation standpoint, they are intimately connected: first the interactive process generates labeled data or "expert demonstration," then this demonstration is used to train a Causal-Intelligence agent that adjudicates claims sans

human input. This section poses this machine learning problem in a precise manner as an instance of *off-policy model-based causal reinforcement learning in the multi-arm bandit setting*. See remark (58) for reinforcement learning decision making regimes.

Game Elements

The origin of reinforcement learning dates back to the 1950s, while causal reinforcement learning using do-Calculus to build more generalizable models is a burgeoning field. A preliminary review of determining causality using do-Calculus is given in section 6.1, sections 6.2.1 and 6.2.2 in particular reviews basic concepts in reinforcement learning (RL) and causal reinforcement learning (CRL), respectively. We will use the notation introduced in the named sections to pose the problem here. In conceptualizing the adjudication process as a RL game, the components are outlined in table (4).

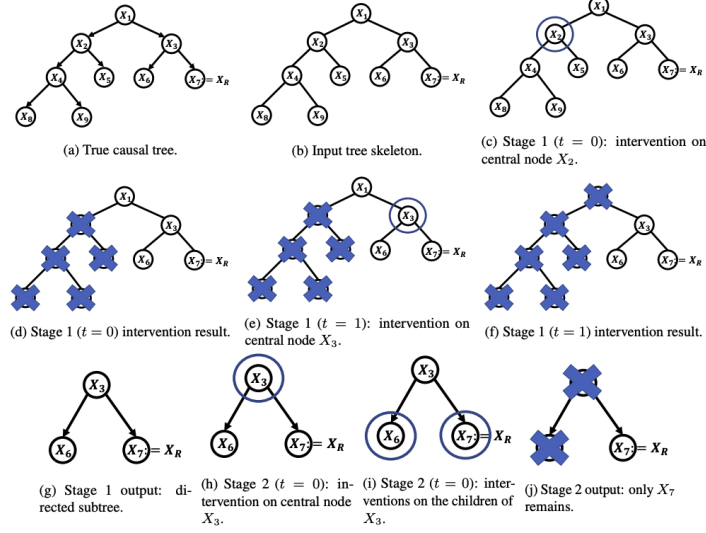
Elements of Reinforcement Learning	AlphaGo	The (pro)Metheus Causal-Intelligence Oracle
The game state s_t	The Go-board over the course of game play	The underlying causal graph $\mathcal{G}$ over the course of interventions
The players	Two Go players	The parties potentially liable for the damaged parcel
The actions a_t	Placing stones on the board	Proposing interventions x_t on the causal graph $\mathcal{G}$
The observations o_t	The current state of the board s_t	The current causal graph post intervention $do(x)$ denoted $p(y \mid do(x))$
State transition kernel	The next game play by the opponent given current play	Does not apply
The intermediate rewards	The likelihood of winning or losing the game	The likelihood of winning or losing the case
The final reward	Winning or losing the game	Winning or losing the case
Game formalization	Markov decision process (MDP)	Multi-arm bandit (MAB)
Game solver	Monte-Carlo tree search (MCTS)	Central-node upper confidence bound (CN-UCB)
State representation/feature extractor	Convolutional neural networks (CNNs)	Sensor data & text processed by open-source large language models (LLMs)
Data set for training	History of Go gameplay	Policy holder Mutual Accountability Application

■ **Table 4** Components of reinforcement learning for the game of Go solved by AlphaGo, and the game of insurance adjudication solved by the (pro)Metheus Causal-Intelligence Oracle [Sil16; Lu21].



Modeling the Game

In the game of Go, there is a board and players place stones on the board to better their position. In the *adjudication game*, the board of engagement or underlying data structure is the input skeleton for a causal graph $\mathcal{G} = (\mathbf{V}, \mathbf{E})$, see fig (13).



■ **Figure 13** The (pro)Metheus Causal-Intelligence Oracle uses variations of CN and CN-UCB algorithm from [Lu21; K19] to find the root cause of an outcome

In $\mathcal{G}$ we have:

- **(The vertices $\mathbf{V}$):** are the factors that contribute to the successful delivery of a parcel. Furthermore, the data access assumptions of section 5.3.1 implies that all the vertices of $\mathcal{G}$ are enumerated and their associated data accessible. Call this set of vertices:

$$\mathbf{V} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}. \quad (16)$$

Now suppose the parcel is created at time $t = 0$, and delivered to the final buyer at time $t = T$. Then for every vertex $\mathbf{x}_i \in \mathbf{V}$, there is an associated timestamped data stream of form:

$$\text{data}(\mathbf{x}_i) = \{(t = 0, x_i^0), (t = 1, x_i^1), \dots, (t = T, x_i^T)\}. \quad (17)$$

Furthermore, there is a final vertex $\mathbf{y}$, which denotes the final condition of the parcel. This $\mathbf{y}$ could be real valued denoting quality on a fine-grained scale, or it could be binary valued $\mathbf{y} \in \{0, 1\}$ denoting whether the parcel was delivered or not.

- **(The directed edges $\mathbf{E}$):** express causation, so that an edge of form $(\mathbf{x}_i, \mathbf{x}_j)$ denotes that changes in the value of $\mathbf{x}_i$ will deterministically induce a change in the value of $\mathbf{x}_j$, modulo exogenous variables that introduce some randomness.

► **Example 38. (Data Stream)** Suppose the factor $\mathbf{x}_{\text{box}}$ is a temperature sensor in a shipping container so that it ranges over $\mathbf{x}_{\text{box}} \in \{-40, 40\}$. Now suppose the parcel was in the shipping container between time $t = 2000$ and $t = 5000$, and let $0 \leq t \leq T$. Then a data stream may be:

$$\text{data}(\mathbf{x}_{\text{box}}) = \{(0, \perp), \dots, (1999, \perp), (2000, -10), \dots, (5000, -12), (5001, \perp), \dots, (T, \perp)\}.$$

Where the $\perp$ symbol denotes no data, as the parcel was not in the container at the time.

Now, whether the adjudication game is played by policy holders in the "expert demonstration" setting, or by the Causal-Intelligence Oracle in the automated setting, the objective is the same:

"They must find the most responsible factor (call it x_R) that causes the outcome y ."

This objective reflects that of the central-node algorithm of 6.2.3 [K19]. Here the authors pose the game of finding the most responsible party as analogues to a gambler in a casino, who pulls on different slot machine arms to determine which arm gives out the best rewards. Over time the gambler becomes proficient and finds the best arm. This is the *multi-arm bandit* (MAB) setting of table 4.

1. **(Solving the game):** the (ρ)Metheus Causal-Intelligence performs structured tree search over the causal graph to output the root cause x_R . Details are found in section 6.2.3, and the process is illustrated in fig 13. [K19] showed that this graph can be recovered in $\log(n)$ number of searches or $\text{do}(x)$ interventions, whereby n is the number of vertices or causal factors in $\mathcal{G}^o$.
2. **(Additional parties at fault):** if the requirement is to enumerate a list of M responsible parties, then it suffices to run the process above M times.
3. **(Expert demonstrations):** observe that the expert demonstration setting is a "manual" version of the same algorithm, albeit without the structured tree search efficiency of it. Repeated run of the expert demonstration or policy holder mutual accountability process will endow (ρ)Metheus with a set of heuristics stating which vertices x_i in the causal DAG $\mathcal{G}$ to try first. In the context of MAB, this is equivalent to equipping the gambler with side information tipping him as to which lever is the most likely to yield some profit.

► Remark 39. **(Truthful elicitation of x_R from policy holders).** When (ρ)Metheus elicits possible causes of a faulty delivery in the mutual accountability application, there is a possibility that the policy holders may collude to point fingers at one party in particular, call this scape-goated party $\hat{x}_R$. There are many ways to resolve this problem:

- The final defense against such malicious behavior is the fact that (ρ)Metheus does not take the suggestion of underlying cause as a fact, but rather determine the likelihood that $\hat{x}_R$ caused y by scoring its intervention against the causal graph.
- *A priori* scoring the suggested $\hat{x}_R$, the (ρ)Metheus application may design mechanisms or incentives so that it is in each policy-holder's interest to report the truth as they *estimate* it.

The exact nature of said mechanism is beyond the scope of this monograph. However the interested reader may reference the Metheus prediction market at [Lin25]¹⁵ for a rich set of examples of how we design incentives.

► Remark 40. **(The Overall (ρ)Metheus Causal-Intelligence System)** goes far beyond the game presented here. It features end user data capture devices, LLM-based end-user interfaces, data sanitation pipelines, and a host of other technologies. However the scope of this monograph is narrow, and focuses on the Causal-Intelligence and Covenant-Compliance aspect of (ρ)Metheus.

5.4 Governing (ρ)Metheus Causal-Intelligence with Loare-Logic

This section concludes the discussion on the (ρ)Metheus Causal-Intelligence Oracle by giving a specific example of how stake-holder-defined Loare-Logic is utilized to audit the process of determining the root cause x_R within the causal graph $\mathcal{G}$. So that when the Causal-Intelligence Oracle operates in

¹⁵ Accessible at: <https://bit.ly/41Tv5T0>

automated mode, it does not arrive at a judgement by ingesting "bad data," and outputting "gibberish" that damages the policy-holder's interests.

In this context, the key is to recall that (pro)Metheus Causal-Intelligence is just software informed by data. Therefore we can approach its validation using the methods introduced in Section 4. This section focuses on a key aspect of algorithm 4 in line 7-9. Persistent auditing of these lines is critical. Because if (pro)Metheus determines that the root cause of a faulty parcel stems from policy-holder $\mathbf{x}_R$, then $\mathbf{x}_R$ will incur future monetary damage in the form of elevated premiums. Repeated misidentification of at-fault-party will damage the reputation (pro)Metheus Causal-Intelligence system, thereby degrading its future prospects. Thus it is in the interest of policy-holders, regulators, as well as (pro)Metheus to audit the process of determining $\mathbf{x}_R$ itself.

The relevant question then is whether the vertex $\mathbf{x}_k$, as well as its neighbors has produced sufficient *quality* data to justify assigning responsibility onto $\mathbf{x}_k$, that is setting $\mathbf{x}_R := \mathbf{x}_k$. We can use Loare-Logic to decorate a program of the expression as follows:

■ **Algorithm 1** CN Algorithm Excerpt, adapted from [K19]

-
- 1: $\left\{ \Gamma \models x_1 : \kappa_2, \dots, x_n : \kappa_n \right\}$
 - 2: Intervene on vertex $\mathbf{x}_o$ and observe sample $\{x_1, \dots, x_n\} / \{x_o\}$
 - 3: Update posterior distribution according to eqn (35)
 - 4: $\left\{ \left\{ \Gamma \models \text{conf}(\mathcal{B}_{\mathcal{G}(t-1)}[\mathbf{x}_o : \mathbf{x}_t]) \geq \kappa^* \right\} \right\}$
 - 5: Using posterior locate neighborhood around $\mathbf{x}_t$ where $\mathbf{x}_R$ is likely to be found
-

Some commentaries follow:

- (line 1): all the data used to adjudicated the outcome has Covenant-Judgement attached with $\Gamma \models x_i : \kappa_i$ for every i . These judgements witness the quality of data used to identify the at fault vertex $\mathbf{x}_R$.
- (line 2): runs an intervention and draws sample $x_1, \dots, x_n$ from the (pro)Metheus data lake. These are data generated from the history of supply chain up to this present moment. This includes the history that is directly relevant to the faulty party under investigation.
- (line 3): uses the dataset drawn to identify the most likely neighborhood in which $\mathbf{x}_R$ resides in the causal tree $\mathcal{G}$.
- (line 4): specify the post condition. It states that the confidence that the root cause is in the neighborhood of $\mathcal{B}_{\mathcal{G}(t-1)}[\mathbf{x}_o : \mathbf{x}_t]$ must be greater than some client or regulator defined value κ^* . This $\text{conf}(\cdot)$ function is a custom write that is built alongside regulator and client demands. Embedding a sufficient number of $\text{conf}(\cdot)$ into the (pro)Metheus Causal-Intelligence workflow is also a *user retention* mechanism, as it means that (pro)Metheus is legally-compliant to a degree that other "agents" cannot match.
- (line 5): if the post condition above is not met at line 3, then algorithm 4 halts at line 4. This prevents the wrong neighborhood of causes/policy holders from being targeted due to faulty data. However if the confidence is greater than κ^* , then (pro)Metheus proceeds to step 5 and continue the search.

After the at-fault party or vertex $\mathbf{x}_R$ has been identified by (pro)Metheus Causal-Intelligence using algorithm 4 or 5, the (pro)Metheus compiler will output a proof tree using the Loare-Logic decoration provided in this program. This tree is similar to example 34, it will be submitted along with $\mathbf{x}_R$ to witness the Covenant-Compliance of the adjudication process.

► Remark 41. (**When a** (ρro) Metheus **program halts**) due to low quality data, (ρro) Metheus could either prompt the user for more data, or hand the adjudication process over to a certified examiner, who will then make a decision and bear the proper risk. This is a sophisticated example of *co-piloting*, albeit with far greater visibility to why the system halts than what vanilla LLMs could offer.

6 Technical Background On Causal Reinforcement Learning

This chapter introduces the relevant aspect of do-Calculus used to infer causality from data. This is followed by causal reinforcement learning (CRL), which for our purpose is a "strategic" way to determine the most likely causal graph.

6.1 Causal Bayesian Networks and do-Calculus

In order to express causation, the language of probability must be augmented with new symbols or algebra to express causal concepts that are intuitive to the human mind. This is the language of do-Calculus applied to the transformation of directed graphical models, it allows (pro)Metheus to estimate causal relations from observational data alone. It does so by translating basic intervention or counterfactual queries into operations on graphical models, questions such as does treatment $\mathbf{x}$ cause outcome $\mathbf{y}$. Finally it uses observational data alone to accept or reject said queries.

6.1.1 Bayesian Network as the Underlying Data Structure

In inferring causality, the elementary data structure of choice to encode the expert's assumptions and/or express relationships found amongst data is the *causal Bayesian network*, which is a specific type of Bayesian network that explicitly models causal relationships between variables. Governing the relationship among the variables are *structural equations*. They are *deterministic* expressions of how the variables under consideration relate to each other. Moreover, we assume the observed uncertainty in outcome arises from external factors that are not account for by the model. This is the Laplace interpretation of probability. This set of parameterized structural equations relating the factors form a *causal model*.

► **Definition 42. (Structural Causal Model)** [Pea09] is a pair $\mathcal{M} = (\mathcal{G}, \mathcal{F}, \mathcal{U})$ consisting of:

- a causal structure $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ is a directed acyclic graph (DAG) where each edge represent some functional relationship amongst the vertices $\mathbf{x} \in \mathbf{V}$.
- A set of endogenous variables $\{\mathbf{x}_1, \dots, \mathbf{x}_i, \dots\}$ inside the model.
- A set of exogenous variables $\mathcal{U} = \{\mathbf{u}_1, \dots, \mathbf{u}_i, \dots\}$ outside the model.
- a set of parameters $\mathcal{F}$ consistent with $\mathcal{G}$. They assign functions:

$$\mathbf{x}_i = f(\text{pa}(\mathbf{x}_i), \mathbf{u}_i),$$

to each vertex $\mathbf{x}_i \in \mathbf{V}$, where $\text{pa}(\mathbf{x}_i)$ are the parent vertices of $\mathbf{x}_i$.

- A probability distribution $\mathfrak{P}(\mathbf{u}_1, \dots)$ over exogenous variables.

► **Example 43. (Structural Causal Model)** [Pea09] An example causal model is presented in fig (14). Its structural equations are:

$$\begin{aligned} \mathbf{x}_1 &\in \{\text{winter, spring, summer, fall}\}, \\ \mathbf{x}_2 &:= f(\text{pa}(\mathbf{x}_2), \mathbf{u}_2) = [(\mathbf{x}_1 = \text{winter}) \vee (\mathbf{x}_1 = \text{fall}) \vee \mathbf{u}_2] \wedge \neg \mathbf{u}'_2, \\ \mathbf{x}_3 &:= f(\text{pa}(\mathbf{x}_3), \mathbf{u}_3) = [(\mathbf{x}_1 = \text{summer}) \vee (\mathbf{x}_1 = \text{spring}) \vee \mathbf{u}_3] \wedge \neg \mathbf{u}'_3, \\ \mathbf{x}_4 &:= f(\text{pa}(\mathbf{x}_4), \mathbf{u}_4) = (\mathbf{x}_2 \vee \mathbf{x}_3 \vee \mathbf{u}_4) \wedge \neg \mathbf{u}'_4, \\ \mathbf{x}_5 &:= f(\text{pa}(\mathbf{x}_5), \mathbf{u}_5) = (\mathbf{x}_4 \vee \mathbf{u}_5) \wedge \neg \mathbf{u}'_5. \end{aligned} \tag{18}$$



■ **Figure 14** Bayesian networks are directed acyclic graphs (DAGs), where vertices represent variables of interest, and edges represent informational or causal dependency amongst the variables. The strength of the dependency is represented by the conditional probabilities labeling each edge. The example graph on the left expresses the various factors that lead to a slippery pavement [Pea09].

Where $\mathbf{x}_i \in \{0, 1\}$, and $\text{pa}(\mathbf{x}_i)$ refer to the parents of variable $\mathbf{x}_i$. Each $\mathbf{u}_k = 1$, with $\neg \mathbf{u}_k = 0$. They are exogenous terms that make the outcome probabilistic. For example if are given these odds:

$$\begin{aligned} p(\mathbf{u}_2) &= 0.6, & p(\neg \mathbf{u}_2) &= 0.4, \\ p(\mathbf{u}'_2) &= 0.2, & p(\neg \mathbf{u}'_2) &= 0.8. \end{aligned}$$

Now given $\mathbf{x}_1$ is winter so that $\mathbf{x}_2 = (\text{winter} \vee \mathbf{u}_2) \wedge \neg \mathbf{u}'_2$. Then we have this contingency table:

$$\begin{aligned} p(\mathbf{x}_2 = 1 \mid \mathbf{x}_1 = \text{winter}) &= p(\text{winter} \vee \mathbf{u}_2 \wedge \neg \mathbf{u}'_2) = p(\text{winter} \wedge \neg \mathbf{u}'_2 \mid \mathbf{x}_1 = \text{winter}) = 0.8, \\ p(\mathbf{x}_2 = 0) &= 0.2. \end{aligned}$$

Suppose instead $\mathbf{x}_1$ is spring so that $\mathbf{x}_2 = (\text{spring} \vee \mathbf{u}_2) \wedge \neg \mathbf{u}'_2$. Then we have this contingency table:

$$\begin{aligned} p(\mathbf{x}_2 = 1 \mid \mathbf{x}_1 = \text{spring}) &= p(\text{spring} \vee \mathbf{u}_2 \wedge \neg \mathbf{u}'_2) = p(\mathbf{u}_2 \wedge \neg \mathbf{u}'_2 \mid \mathbf{x}_1 = \text{spring}) = 0.6 \times 0.8 = 0.48 \\ p(\mathbf{x}_2 = 0) &= 0.52. \end{aligned}$$

The exact set of equations specifying the causal relations is not important here, what is salient is that structural equations are deterministic relations that govern the outcome of variables. Any probability distribution defined over $\mathbf{x}_1, \dots, \mathbf{x}_5$ is over the uncertain values of $\mathbf{u}_k$, which form the source of randomness. Without the $\mathbf{u}$ factors, the values of $\mathbf{x}_i$ is completely determined [Pea09].

Similar to conventional Bayes networks, given a model $\mathcal{M}$ its likelihood is factorized with:

$$p(\mathbf{x}_1, \dots) = \prod_i p(\mathbf{x}_i \mid \text{pa}(\mathbf{x}_i)). \quad (19)$$

The entries of each $p(\mathbf{x}_i \mid \text{pa}(\mathbf{x}_i))$ may be computed according to example (43) above.

6.1.2 Operation on Data Structure with do-Calculus

We can query the graph in figure 14 with the three questions on the ladder of causation of table 3:

1. *Prediction*: would the pavement be slippery if *see* the sprinkler is off? That is we query the probability $p(\mathbf{x}_4 = \text{Slippery} \mid \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3 = \text{Off}, \mathbf{x}_4)$.
2. *Intervention*: would the pavement be slippery if we *turn* the sprinkler on? The graph to the right expresses such an intervention whereby the underlying state is set to "sprinkler=ON" regardless of

the condition of the season, which under observed data may affect the status of sprinkler under non-intervened conditions. Note the edge connecting the season vertex to the sprinkler vertex has been pruned as the sprinkler variable is now a constant. That is to say we are computing the probability $p(\mathbf{x}_5 = \text{Slippery} \mid \mathbf{x}_1, \mathbf{x}_2, \text{do}(\mathbf{x}_3 = \text{On}), \mathbf{x}_4)$.

3. *Counterfactual*: would the pavement be slippery had the sprinkler been off, given we observe the pavement is not slippery and the sprinkler is on? That is we query the likelihood $p(\mathbf{x}_5 = \text{Slippery} \ \& \ \mathbf{x}_3 = \text{Off} \mid \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3 = \text{On}, \mathbf{x}_4, \mathbf{x}_5 = \neg\text{Slippery})$.

This paper focuses on the second rung of the causal ladder: discovering causal factors by intervention. Since the structural equations of eqn (18) determine a "fixed mechanical system," one can simulate, i.e. intervention of form "turning the sprinkler on," written:

$$\text{do}(\mathbf{x}_3 = \text{On}),$$

by removing equation $\mathbf{x}_3 = f(\text{pa}(\mathbf{x}_3), \mathbf{u}_3)$ to create the "mutilated graph" of the right picture in fig 14. This pruned graph will be used to determine if the sprinkler variable $\mathbf{x}_3$ *causes* the pavement to be wet.

► **Definition 44. (Causal Effect)[Pea09]** Given two variables $\mathbf{x}$ and $\mathbf{y}$ in some causal model $\mathcal{M}$, the causal effect of $\mathbf{x}$ on $\mathbf{y}$, denoted with:

$$p(\mathbf{y} = y \mid \text{do}(\mathbf{x} = x)), \quad (20)$$

is a function from $\mathbf{x}$ to the space of probability distributions on $\mathbf{y}$. Eqn (20) gives the probability of $\mathbf{y} = y$ induced by deleting from the causal model $\mathcal{M}$ all structural equations leading to $\mathbf{x}$, and substituting $\mathbf{x} = x$ in the remaining equations in $\mathcal{M}$. Alternatively, one can define the difference:

$$E_{\mathbf{u}}[\mathbf{y} \mid \text{do}(\mathbf{x} = x_1)] - E_{\mathbf{u}}[\mathbf{y} \mid \text{do}(\mathbf{x} = x_2)], \quad (21)$$

as the causal or "average treatment effect" of the $\text{do}(\mathbf{x})$ operation.

► **Example 45. (Causal Effect)** Continuing example of fig (14) in example 43, a new graph where $\mathbf{x}_3$ is set to On is parameterized by the causal model:

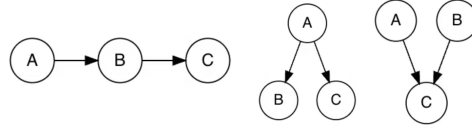
$$\begin{aligned} \mathbf{x}_1 &\in \{\text{winter}, \dots\}, & \mathbf{x}_2 &:= f(\text{pa}(\mathbf{x}_2), \mathbf{u}_2), & \mathbf{x}_3 &:= \text{On}, \\ \mathbf{x}_4 &:= f(\text{pa}(\mathbf{x}_4), \mathbf{x}_3 = \text{On}, \mathbf{u}_4) = (\mathbf{x}_2 \vee \text{On} \vee \mathbf{u}_4) \wedge \neg \mathbf{u}_4', & \mathbf{x}_5 &:= f(\text{pa}(\mathbf{x}_5), \mathbf{u}_5). \end{aligned} \quad (22)$$

Note that the sprinkler variable $\mathbf{x}_3$ no longer depends on $\mathbf{x}_1$, or the seasons. Meanwhile, in equation $\mathbf{x}_4$ the variable $\mathbf{x}_3$ is set to On [Pea09].

Once the edges incident upon some $\mathbf{x}$ in the causal model have been deleted and its value set to $\mathbf{x} = x$, we can *identify* whether this factor $\mathbf{x}$ is an actual cause of some downstream effect denoted $\mathbf{y}$. In the absence of experimental data, this is done by computing the *post-intervention probability* using observed data from the *pre-intervention probabilities*. The exact nature of the post-intervention probability after $\text{do}(\mathbf{x})$ adjustment depend on the local topology of the graph around the $\mathbf{x}$ variable. Often, additional adjustments must be made to the graph before the post-intervention likelihood is computed. The three rules of do-Calculus makes these adjustments in a principled manner:

► **Theorem 46. (Rules of do-Calculus)[Pea09; Pea12]**

1. **(Ignoring observations)**: that is if we wish to determine if $\mathbf{x}$ causes $\mathbf{y}$ after applying $\text{do}(\mathbf{x})$, then these vertices may be deleted from the causal graph:
 - vertices with no path to $\mathbf{y}$.
 - vertices that is d -separated from the outcome $\mathbf{y}$. See theorem 47 below.



■ **Figure 15** Three types of separation patterns are chain, fork, and collider. In each case, if the middle vertex in the graph is determined then the path is broken, and the two distal variables are no longer correlated [25c].

- (Left): in the chain pattern or "mediation," **b** separates **a** from **c**, s.t. the value of **c** is independent of **a** given **b**
- (Middle): in the fork pattern or "mutual dependence," **a** separates **b** from **c**, s.t. the value **c** is independent of **b** given the value of **a**.
- (Right): the collider pattern or "mutual causation," **c** separates **a** from **b**, s.t. the values of **a** and **b** are independent given **c**.

Name this vertex **z**. Now suppose all the edges incident on **x** are removed, then this rule is written:

$$p(y \mid z, \text{do}(x), w) = p(y \mid \text{do}(x), w) \quad \text{if} \quad y \perp\!\!\!\perp z \mid w, x. \quad (23)$$

Stating that if variable **z** is independent of outcome **y** given the preset values of **x** and **w**, then **z** may be removed from the graph.

2. (**Treating interventions as observations**): this rule states that interventions of form $\text{do}(x)$ on outcome **y** can be treated as an observation when the causal effect of **x** on **y** only influences the outcome through directed paths. Suppose we wish to test the causal effect of variable **z** on **y**, then given a mutilated graph where all the edges going out of **z** are removed, we have:

$$p(y \mid \text{do}(x), \text{do}(z), w) = p(y \mid \text{do}(x), z, w) \quad \text{if} \quad y \perp\!\!\!\perp z \mid w, x. \quad (24)$$

3. (**Ignoring interventions**): sometimes one can ignore an intervention of form $\text{do}(z)$ on **y** if **z** does not influence the outcome **y** through any uncontrolled paths. That is to say there is no associational path from **z** to **y**. Now given a graph whereby all edges incident on **x** has been removed, and all edges outgoing from **z** has been removed, then we have:

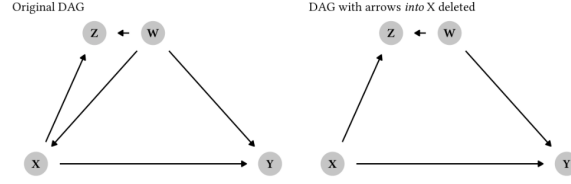
$$p(y \mid \text{do}(x), \text{do}(z), w) = p(y \mid \text{do}(x), w) \quad \text{if} \quad y \perp\!\!\!\perp z \mid w, x. \quad (25)$$

► **Theorem 47.** (*d-separation*) Given a causal graph with vertices **V**, a set $\mathbb{S} \subseteq \mathbf{V}$ of vertices is said to block a path from $x \in \mathbf{V}$ to $y \in \mathbf{V}$ if either one of the follow two conditions are true:

1. The path contains at least one arrow-emitting vertex that in the set $\mathbb{S}$. This is the middle vertex in a chain or fork pattern. See figure (15).
2. The path contains at least one collision vertex that is outside $\mathbb{S}$, and has no descendant in $\mathbb{S}$. This is the middle vertex in a collision pattern, see figure (15).

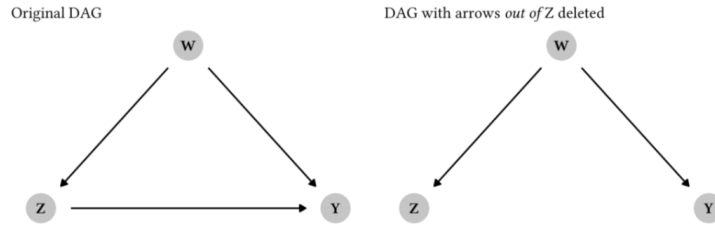
If the set of vertices $\mathbb{S}$ blocks all path from **x** to **y**, it is said to "d-separate **x** and **y**." Two sets $\mathbb{A} \subseteq \mathbf{V}$ and $\mathbb{B} \subseteq \mathbf{V}$ are said to be d-separated by $\mathbb{S}$ if any pair $x \in \mathbb{A}$ and $y \in \mathbb{B}$ are d-separated by $\mathbb{S}$. If so, then we have the following property: $\mathbb{A} \perp\!\!\!\perp \mathbb{B} \mid \mathbb{S}$.

► **Example 48.** (Rule 1)[25c]

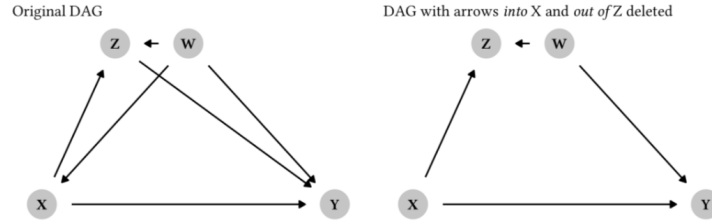


■ **Figure 16** (Left): the original DAG with causal functions incident on x from w . (Right): we query the graph for $p(y \mid do(x), z, w)$. Since the vertex triple (z, w, y) form the "mutual independence" pattern of fig (15), we have the reduced post-intervention distribution $p(y \mid do(x), z, w) = p(y \mid do(x), w)$ by rule one.

► **Example 49. (Rule 2)[25c]**

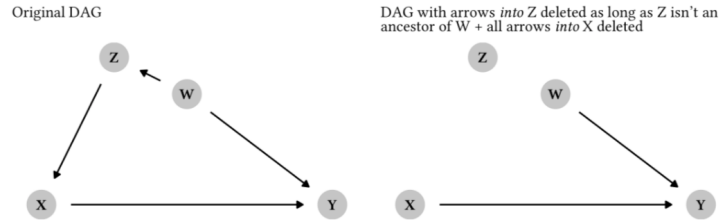


■ **Figure 17** (Left): the original graph where we wish to query z for its causal effect on y . (Right): the mutilated graph where edges going out of z has been removed. Note that this is the fork pattern, so we can write $p(y \mid do(z), w) = p(y \mid z, w)$ by rule 2.



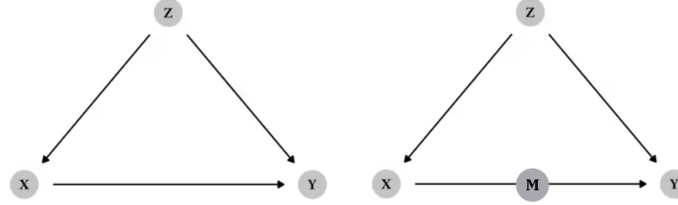
■ **Figure 18** (Left): the original graph where we wish to query z for its causal effect on y . (Right): the mutilated graph where edges going out of both x and z have been removed. Now observe the fork pattern with the triplet (z, w, y) where $y \perp z \mid w$, so we can write $p(y \mid do(z), do(x), w) = p(y \mid do(x), z, w)$ by rule 2.

► **Example 50. (Rule 3)[25c]**



■ **Figure 19** (Left): original graph. (Right): $do(x)$ is applied s.t. when the edges incident on x are removed, z does not influence y in any way, so that its edges are deleted via $p(y \mid do(x), do(z), w) = p(y \mid do(x), w)$.

Using the three algebraic rules of do-Calculus above, we can derive the following two expressions that turn any post-intervention distribution into observational distributions, so that the effect of an intervention can be identified from observational data alone.



■ **Figure 20** (Left) Back-door criterion: here z is a confounding variable influencing both x and y , so that x is independent from y given z . (Right) front-door criterion, where z is an *unobserved* confounding variable influencing both x and y , while m is an observed variable blocking the path from x to y .

► **Proposition 51. (Back-door Criterion)**[Pea09] *Given the left causal graphical model in fig (20), we wish to query the graph for the effect of treatment x on outcome y by $\text{do}(x)$. Then we can reduce the query as follows:*

$$p(y \mid \text{do}(x)) = \sum_z p(y \mid x, z) p(z). \quad (26)$$

Proof.

$$\begin{aligned} p(y \mid \text{do}(x)) &\stackrel{(1)}{=} \sum_z p(y \mid \text{do}(x), z) p(z \mid \text{do}(x)) \stackrel{(2)}{=} \sum_z p(y \mid x, z) p(z \mid \text{do}(x)) \\ &\stackrel{(3)}{=} \sum_z p(y \mid x, z) p(z \mid \cdot) = \sum_z p(y \mid x, z) p(z). \end{aligned}$$

Where going into (1) we use law of total probability and the chain rule. Going into (2), we use rule 2 to treat $\text{do}(x)$ as x . Going into (3) we use rule 3 to ignore $\text{do}(x)$. ◀

► **Proposition 52. (Front-door Criterion)**[Pea09] *Given the right causal graphical model of fig (20), the graphical query for the effect of treatment x on outcome y can be reduced with:*

$$p(y \mid \text{do}(x)) = \sum_m p(m \mid x) \sum_{x'} p(y \mid x', m) p(x'). \quad (27)$$

Proof.

$$\begin{aligned} p(y \mid \text{do}(x)) &\stackrel{(1)}{=} \sum_m p(y \mid \text{do}(x), m) p(m \mid \text{do}(x)) \\ &\stackrel{(2)}{=} \sum_m p(y \mid \text{do}(x), \text{do}(m)) p(m \mid x) \\ &\stackrel{(3)}{=} \sum_{m,z} p(y \mid \text{do}(x), \text{do}(m), z) p(z \mid \text{do}(x), \text{do}(m)) p(m \mid x) \\ &\stackrel{(4)}{=} \sum_{m,z} p(y \mid \text{do}(m), z) p(z \mid \text{do}(m)) p(m \mid x) \\ &\stackrel{(5)}{=} \sum_m p(y \mid \text{do}(x)) p(m \mid x) \stackrel{(6)}{=} \sum_m p(m \mid x) \sum_{x'} p(y \mid \text{do}(m), x') p(x' \mid \text{do}(m)) \\ &\stackrel{(7)}{=} \sum_m p(m \mid x) \sum_{x'} p(y \mid m, x') p(x' \mid \text{do}(m)) \stackrel{(8)}{=} \sum_m p(m \mid x) \sum_{x'} p(y \mid m, x') p(x'). \end{aligned}$$

Where going into (1) and (2), we applied rule 2. Going into (3) we applied rule 3. Going into (4) we introduce unobserved variable z and marginalize over it. Going into (5) and (6) we introduce x' and marginalize it out. Going into (7) and (8) we applied rules 2 and 3. ◀

If repeated application of do-Calculus rules allows one to convert an expression with $do(x)$ operator to one that is $do(x)$ -free akin to eqn (26) and (27), then the causal effect of x on the outcome y is *identifiable*.

► **Example 53. (Answering the Forward Question)** Once the rules of do-Calculus are established, the forward question is trivially answered in the following setting:

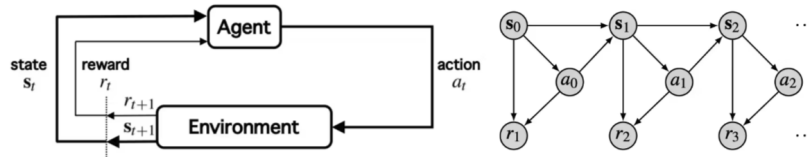
1. The analyst is given data set and structural causal model $\mathcal{M}$ in some (pro)Metheus environment.
2. The analyst queries $\mathcal{M}$ with i.e. $do(x_1)$ to identify the causal effect of x_1 on y .
3. The (pro)Metheus Typed ρ -Calculus reduces the query using do-Calculus, and computes the likelihood of $p(y \mid do(x_1))$. This likelihood is a score denoting the how much x_1 influence y .
4. Other analysts may then repeat this procedure for other factors $\{x_2, \dots, x_i, \dots\}$.
5. The (pro)Metheus oracle then ranks the likelihood of $p(y \mid do(x_i))$ to establish the cause of outcome y w.r.t. each x_i .
6. If however $p(y \mid x_i)$ is not identifiable for some x_i , then the (pro)Metheus program terminates with either:
 - elicitation for more data;
 - or request for expert input in adjusting the assumptions of the graph. That is the edges or functions $f(pa(x_i), u_i)$ of the structural causal model.

This aspect is important: request for more data is a *value driver* in the underlying commodity/data economy of the (pro)Metheus ecosystem. While request for expert advice is a *growth driver* for new user acquisition.

Observe that in many circumstances, the complete dataset may not be revealed to all parties due to privacy and/or legal concerns. In other settings, the data is revealed under differential privacy masks which may reverse the ranking of causal factors. In these cases, the (pro)Metheus oracle will act as the "neutral chamber" that adjudicates the outcome based on access to all the information.

6.2 Causal Reinforcement Learning

While the forward question may be trivially answered given sufficient data and the identifiable criteria is met, the reverse direction is categorically more complex to answer. In the backward question setting, the (pro)Metheus oracle is given raw data, and must search for the most likely causal graph *tabula rasa*. This falls in the domain of machine learning, specifically reinforcement learning (RL).



■ **Figure 21** Reinforcement learning defines how an autonomous Agent learns to what to do in order to maximize a collection of sequential numerical rewards. This is an interactive setting whereby an Agent acts on the Environment at each time step t with action a_t . This action a_t changes its underlying state from s_t to s_{t+1} . The Agent observes this delta and sometimes receives a reward r_t at each t , other times the Agent may only receive a final reward when the game terminates at time T [Sut15].

6.2.1 Reinforcement Learning and MDPs

RL is different from supervised learning (i.e.: classification and regression) in that this setting does not provide labeled data. Instead, it is a form of goal-directed learning from repeated interactions with some Environment [C22; Bar25]. Since the RL Agent plays to win, the strategy of how to play the game will be of vital importance, not only for maximizing rewards, but also for computational efficiency. Given some Agent in an Environment, the game proceeds as follows:

1. First the Agent takes an action a_t .
2. Then the Environment responds by:
 - giving the Agent a reward r_t ,
 - and increments the state from s_t to s_{t+1} .

So that as the learner must explore the Environment, it constructs an observed state space of form:

$$\{\dots, (s_t, a_t, r_t, s_{t+1}), \dots\}. \quad (28)$$

The output of reinforcement learning algorithms are two-fold:

1. A strategy for how to play "the game" to maximize the reward.
2. Sometimes, the algorithm will also output an underlying world model based on the state space it has explored in (28).

The (pro)Metheus oracle constructs this underlying world model as a structural causal model used to assign liability to various parties.

► **Definition 54. Markov Decision Process (MDP)**[Bar25] is a tuple $(\mathbf{S}, \mathbf{A}, p(\cdot), \mathcal{R}, \gamma)$ where:

- $\mathbf{S}$: is a state space which contains all possible states of the Environment.
- $\mathbf{A}$: action or policy space containing all possible actions the Agent could take.
- $p(s_{t+1} | s_t, a_t)$: is a stochastic transition kernel which gives the likelihood of reaching s_{t+1} from s_t if the Agent takes action a_t . Observe the next state s_{t+1} is independent of all past states $s_0, \dots, s_{t-1}$, conditioned on current state s_t and action a_t . This is the "Markov" aspect of MDP.
- $\mathcal{R} : \mathbf{S} \times \mathbf{A} \rightarrow \mathbb{R}$: a reward function that given state and action, outputs real valued reward.
- $\gamma \in [0, 1]$: a discount factor that down-weighs future rewards.

The MDP sequence is depicted on the right in fig (21). The goal of RL is to derive a policy:

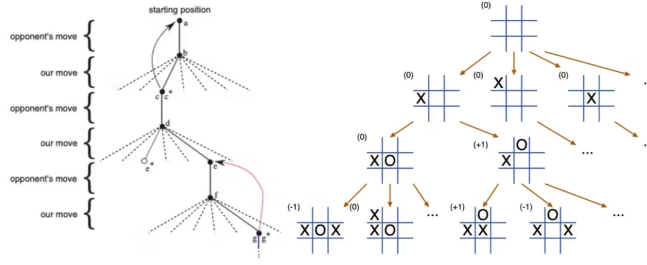
$$\pi(\cdot) : \mathbf{S} \rightarrow \mathbf{A}$$

that maximizes the cumulative reward across time-steps:

$$\mathbb{E} \left[\sum_t \gamma^t \mathcal{R}(s_t, a_t) \mid s_0 = s_k \right] \quad \text{where } a_t \sim \pi(s_t). \quad (29)$$

Here the expectation is taken over all possible sequences of gameplay the Agent may encounter. In the event where the Agent is playing some game against an opponent, then implied in the possible game sequences are the set of possible moves the opponent could make.

► **Remark 55. (Managing State Space in RL)** In a simple game such as tic-tac-toe with 9 possible positions on the board, the state space is manageable. In a more complex games such as Go with hundreds squares, the raw state space of the game becomes intractable. In this case machine learning



■ **Figure 22 (Right):** RL game tree for tic-tac-toe, where the state s_t is the current positions on the game board, the actions a_t are where to place the X or O's, the reward comes at time T when the game ends and the winner is rewarded. Board games are MDPs because the board is plainly visible. In contrast card games (i.e. poker) are POMDPs because the player cannot observe the opponents hands, but instead keeps a distribution over all possible hands the opposite may hold. (Left): a game tree traced by an MDP. The solid lines represent the moves taken during a game; the dashed lines are moves that the player considered but did not make [J P17].

techniques are needed to compress the representation of the state space, this was the part of the breakthrough of Alpha-Go [Sil16].

If the underlying state is completely observed, then this gameplay is a *Markov Decision Process* (MDP), which formalizes the RL setting. See definition 54. If however the underlying state is only partially accessible, then the setting is a *Partially Observed Markov Decision Process* (POMDP). Here the Agent is not playing against a particular state, but a distribution over all possible states. Thus computing the best move may be intractable unless the state space itself is parsimoniously represented. Once again, neural networks must be used to compress the state space. Another option that further reduces the size of the state space is causal reinforcement learning. It uses structural invariants in the Environment to find a better representation of the world that is smaller and more robust.

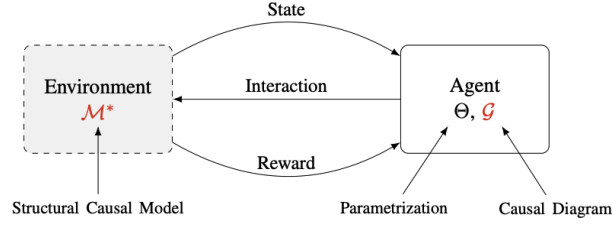
► **Remark 56. (Online vs offline learning)** are two different settings in RL w.r.t when and how the Environment is shown to the Agent.

- In *online learning*, the Agent is interacting with the Environment in real time, and collect reward whilst exploring its surroundings.
- In *off-policy learning*, the Agent learns an optimal policy from offline data generated by a different behavior policy or agent. This is also known as *imitation learning* or *learning from experts*. This is the setting of the (pro)Metheus oracle.

Reinforcement learning is a general setting that applies to many domains with various underlying states. The underlying state could be a game board in Alpha-Go. It may be a mechanical system, i.e. a robot that hops on one leg. In the case of (pro)Metheus, the underlying state is a world model, whereby the observed data are generated by a set of causal relations waiting to be discovered.

6.2.2 Causal Reinforcement Learning with Known Causal Graph

Causal reinforcement learning (CRL) was formalized to address the fact that although statistical models "should" perform out of sample, they often fail to generalize. One avenue towards more robust models is to build ones that express structural invariances in the world. This entails learning a causal model so that the AI can reason about questions of counterfactual nature, i.e., what would have happened had reality been different, even when no data about this imagined reality is available, and the Environment is not fully observable. In CRL, this causal underlying reality is explicitly expressed.



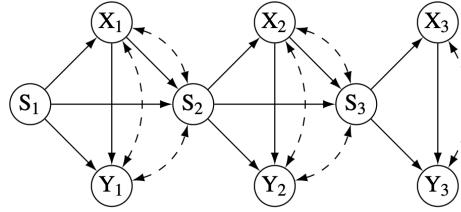
■ **Figure 23** The Agent-Environment interaction from Causal Reinforcement Learning [Bar25].

There is an intrinsic duality between how the Agent and Environment view causality as seen in fig (23):

- From the Environment's perspective, causal mechanisms and the probability distribution over the exogenous variables are described by the structural causal model $\mathcal{M}^*$ on the left side. This is a platonic ideal that is assumed to exist.
- From the Agent's perspective, a parsimonious representation of the Environment's invariances is encoded in the form of a causal graph $\mathcal{G}$ on the right side [Bar25].

The (pro)Metheus oracle builds this causal graph $\mathcal{G}$ using CRL by undertaking a causal discovery process, expressed by the *causal decision model*.

► **Definition 57. Causal Decision Model (CDM)** [Bar25]



■ **Figure 24** The MDP of fig (21) re-expressed as a CDM, where the Agent acts on the Environment with actions $a_t := x_t$, thus changing the Environment state from s_t to s_{t+1} , yielding an observation of y_t [Bar25].

CDM is a tuple

$$(\mathcal{M}^*, \Pi, \mathcal{R}), \quad \text{where } \mathcal{M}^* = (\mathcal{G}, \mathcal{F}, \mathcal{U}), \quad (30)$$

Over decision horizon or number of rounds $1, \dots, t, \dots, T$. And over variable spaces:

$$\begin{aligned} (\text{state space}) : & \quad \mathbf{S} = \{s_1, \dots, s_T\}, \\ (\text{action/intervention space}) : & \quad \mathbf{x} = \{x_1, \dots, x_T\}, \\ (\text{treatment effect space}) : & \quad \mathbf{y} = \{y_1, \dots, y_T\}. \end{aligned} \quad (31)$$

So that:

- $\mathcal{M}^*$ is the true underlying structural causal model.

- Π is a policy space denoted:

$$\{(x_1, s_1), \dots, (x_T, s_T)\}, \text{ or } \pi_t(x_t | s_t) \forall t \quad (32)$$

Such that each action x_t is non-descendent of $x_{t+1}, \dots, x_T$. And each state s_t is non-descendent of $x_t, \dots, x_T$. See example (59) below.

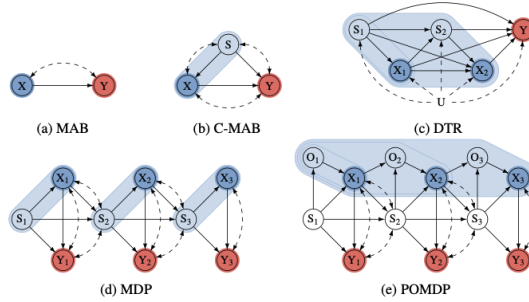
- $\mathcal{R}$ is a reward function over effect signals $y = \{y_1, \dots\}$ defined with:

$$\mathcal{R} : y \rightarrow \mathbb{R}, \quad (33)$$

mapping treatment effect to real valued reward. See fig (24).

In general, we focus on MDPs and POMDPs, however it is helpful to note that there is an escalation ladder in complexity of planning the Agent is asked to undertake. This ladder is defined w.r.t the planning horizon.

► Remark 58. (**CRL Learning Regime $\mathcal{L}$**). The decision Horizon T and ability to perceive confounding variables define the kind of game, or learning regime $\mathcal{L}$ the Agent is playing.



■ **Figure 25** Causal reinforcement learning may be represented by various decision making processes [Bar25].

- (a) In the simplest *multi-arm bandit* setting, the Agent pulls a lever with action x_t and receives a reward y . The policy space is $\Pi = \{(x, \emptyset)\}$, so that Agent simply samples some treatment value x from the the distribution by $x \sim \pi(x)$. The reward is just $\mathcal{R}(y)$.
- (b) In the *contextual bandit* setting with confounding variable s , the policy space is $\Pi = \{(x, \{s\})\}$, and the Agent sample policy by $x \sim \pi(x|s)$. The reward is once again $\mathcal{R}(y)$.
- (c) In the *dynamic treatment regime* setting, the policy space is:

$$\Pi = \{\pi_1(\cdot), \dots, \pi_T(\cdot)\} = \{(x_1, \{s_t, \dots, s_t, x_1, \dots, x_{t-1}\})\}_{t=1}^T.$$

So that the current treatment x depends on past treatments, as well as all the confounding variables or system states s_t 's up to this time step t . Each treatment x_t is now sampled out of the policy:

$$x_t \sim \pi_t(x | s_1, \dots, s_t, x_1, \dots, x_{t-1}).$$

The reward is: $\sum_t^T \mathcal{R}(y_t)$

- (d) *Markov Decision Process* is a generalization of dynamic treatment regime, whereby the Environment has infinite states (confounding variables), actions, and observations. And we have:

$$\Pi = \{(x_t, \{s_1, \dots, s_t, x_1, \dots, x_{t-1}\})\}_{t=1}^\infty, \text{ and } x_t \sim \pi_t(x | s_1, \dots, s_t, x_1, \dots, x_{t-1}).$$

The reward is now discounted by $\sum_t^\infty \gamma^t \mathcal{R}(y_t)$.

- ─ (e) In *partially observed Markov decision process*, the Agent cannot access the underlying state directly, but instead perceives it with observation variable $\mathbf{o}$. So that we have:

$$\Pi = \{(\mathbf{x}_t, \{\mathbf{o}_1, \dots, \mathbf{o}_t, \mathbf{x}_1, \dots, \mathbf{x}_{t-1}\})\}_{t=1}^{\infty}, \text{ and } \mathbf{x}_t \sim \pi_t(\mathbf{x} | \mathbf{o}_1, \dots, \mathbf{o}_t, \mathbf{x}_1, \dots, \mathbf{x}_{t-1}).$$

The reward is $\sum_t \gamma^t \mathcal{R}(y_t)$.

- **Example 59. (DTR Policy Space)**[Bar25] A treatment regime or policy space in healthcare for alcoholics may be expressed with:

$$\mathcal{M}^* = \left(\mathcal{U} = (\mathbf{u}_1, \dots, \mathbf{u}_5), \mathcal{V} = (s_1, s_2, x_1, x_2, \mathbf{y}), \mathcal{F}, \mathbb{P}(\mathcal{U}) \right).$$

Where:

- ─ $\mathcal{U}$ are the exogenous variables outside of the model.
- ─ $\mathcal{V}$ are the endogenous variables inside the model, with interpretations:
 - ─ s_1 : not sober
 - ─ s_2 : sober
 - ─ x_1 : behavioral therapy
 - ─ x_2 : drug treatment
 - ─ $\mathbf{y}$: days sober.
- ─ $\mathcal{F}$ is encoded with:

$$\begin{aligned} s_1 &\rightarrow \mathbb{I}(\mathbf{u}_3 > 0), \\ x_1 &\rightarrow \mathbb{I}(3s_1 + \alpha_1 \mathbf{u}_1 + \mathbf{u}_2 > 0), \\ s_2 &\rightarrow \mathbb{I}(0.1 + 0.1s_1 + 0.1x_1 + \mathbf{u}_4 > 0), \\ x_2 &\rightarrow \mathbb{I}(3s_2 + \alpha_2 \mathbf{u}_1 + \mathbf{u}_3 > 0), \\ \mathbf{y} &\rightarrow \mathbb{I}(3\mathbf{u}_1 - 3s_1 - 3x_1 - 3s_1x_1 + 3x_2 - 3s_2x_2 + 3x_1x_2 > 0). \end{aligned}$$

- ─ $\mathbb{P}(\mathcal{U})$ is a distribution over the exogenous variables defined with:

$$p(\mathbf{u}_t < u) = \frac{1}{1 + e^{-u}}.$$

Now we may construct a restricted policy space or treatment regime as follows:

$$\Pi_1 = \left(\pi_1(x_1 | s_1), \pi_2(x_2 | s_2) \right).$$

One could also adopt a dynamic or responsive treatment regime with:

$$\Pi_2 = \left(\pi_1(x_1 | s_1), \pi_2(x_2 | s_1, x_1, s_2) \right).$$

And observe that Π_1 does not account for whether treatment $x_1 :=$ behavioral therapy was prescribed first, where as Π_2 does account for past treatment and responses.

Example 59 can be generalized into the *causal reinforcement learning task*.

- **Definition 60. (Causal Reinforcement Learning Task)** [Bar25] For an SCM $\mathcal{M}^* = (\mathcal{G}, \mathcal{F}, \mathcal{U})$, a CRL task $\mathcal{T}$ in Environment $\mathcal{M}^*$ is a 4-tuple $(\mathcal{L}, \mathcal{A}, \Pi, \mathcal{R})$ where:

- ─ $\mathcal{L}$ is the Agent's learning regime as seen in remark (58).

- $\mathcal{A}$ is a set of structural assumptions about SCM $\mathcal{M}^*$.
- Π is the policy space over actions $\mathbf{x}$.
- $\mathcal{R}$ is the reward function over effect signals $\mathbf{y}$.

Similar to RL, the Agent's goal is to find the optimal policy w.r.t the underlying causal structure $\mathcal{M}^*$:

$$\pi^* = \arg \max_{\pi \in \Pi} \mathbb{E}_{\pi} \left[\text{Reward}(\mathbf{y}) \mid \mathcal{M}^*, \mathcal{A}, \mathcal{L} \right]. \quad (34)$$

And observe π^* may be different depending on whether Agent is in (C)MAB, DTR, or (PO)MDP setting, as these are given in the learning regime $\mathcal{L}$.

Every causal RL task proceeds with the following computational skeleton.

■ **Algorithm 2** Causal Reinforcement Learning Agent [Bar25]

Require: CRL task $\mathcal{T} = (\mathcal{L}, \mathcal{A}, \Pi, \mathcal{R})$.

Ensure: a policy estimate $\pi^* \in \Pi$ optimizing a CDM $(\mathcal{M}^*, \Pi, \mathcal{R})$.

- 1: $\mathbb{D} \leftarrow \{\}$
- 2: **for** $t = 1 \dots T$ **do**
- 3: Interact with SCM $\mathcal{M}^*$ following regime $\mathcal{L}$ and receive treatment response:

$$y_t \sim p(\mathbf{y} \mid \mathcal{M}^*).$$

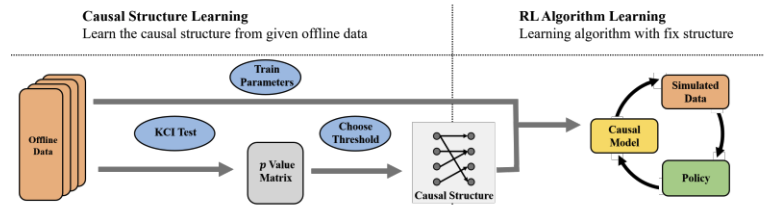
- 4: $\mathbb{D} \leftarrow \mathbb{D} \cup \{y_t\}$
- 5: **end for**
- 6: **return** empirical estimate $\pi \in \Pi$ for π^* from data $\mathbb{D}$ and assumptions $\mathcal{A}$.

In line 3 above, the Agent interacts with $\mathcal{M}^*$ under the (PO)MDP regime as follows:

1. Perceive state s_t or observation o_t .
2. Select an action x_t with $x_t \sim \pi_t(x_t | s_t)$ and perform $\text{do}(\mathbf{x} = x_t)$.
3. Receive treatment response y_t based on $\text{do}(\mathbf{x})$ action.

6.2.3 Causal Reinforcement Learning with Unknown Causal Graph

The algorithm above presumes the Agent is equipped with set of assumptions $\mathcal{A}$ about the underlying causal structure of the world model $\mathcal{M}^*$. This is a fair assumption in many settings. In this section we consider the problem whereby the Agent must learn a causal model *tabula rasa*, with $\mathcal{A} = \emptyset$. This is the backward question of causality as defined in section 5.2.1.



■ **Figure 26** The backward causation problem can be solved by CRL by breaking the process into two steps: 1) a causal graph $\mathcal{G}$ discovery process, and 2) and CRL strategy π optimization process [ZM22].

The computational skeleton for CRL with unknown causal graph proceeds as below.

■ **Algorithm 3** Offline CRL with Unknown Graph Structure (Adapted from [ZM22])

Require: Dataset $\mathbb{D}$

Require: RL learning parameters $(\mathcal{L}, \Pi, \mathcal{R})$

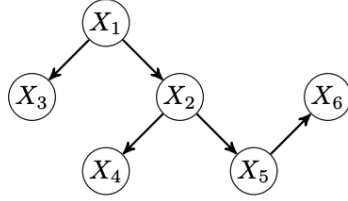
- 1: **Stage 1** Causal structure learning
- 2: $\mathcal{G} \leftarrow \text{learnGraph}(\mathcal{L}, \Pi, \mathcal{R}, \mathbb{D})$
- 3: **Stage 2** Offline reinforcement learning
- 4: Define $\mathcal{A}$ from $\mathcal{G}$
- 5: $\pi \leftarrow \text{Algorithm1}(\mathcal{L}, \mathcal{A}, \Pi, \mathcal{R})$
- 6: **return** policy and causal model pair $(\pi, \mathcal{G})$

Learning the causal graph $\mathcal{G}$ *tabla rasa* is nontrivial, as the greedy approach whereby each vertex $\mathbf{x}$ is independently explored as a causal factor is exponentially suboptimal in the number of interventions required [K19]. Here we review an example algorithm that explores the causal factors given certain (mild) assumptions on the structure of true causal graph $\mathcal{G}^*$.

Central Node: Sample Efficient Active Learning of Causal Trees

- **(Summary).** In [K19], the authors design the Central-Node (CN) algorithm that adaptively determine the root cause of an outcome given a set of potentially causal factors. CN operates in a setting where observation data is cheap, while intervention data is expensive. It is a Bayesian algorithm that initially selects a random cause for intervention, and then adaptively determines the next cause to intervene on, given the outcome of the all the previous interventions.
- **(Objective).** Given an undirected tree that does not contain any unshielded colliders (see fig (27)), design experiments adaptively to learn the underlying causal graph with the minimum expected number of interventions. In this context, expectation is with respect to a given prior distribution over all causal graphs with the given tree as their essential graph.
- **(Network Centrality Measure).** CN finds the root cause of within a causal graph $\mathcal{G}$ by computing the *network centrality* of select vertices $\mathbf{x}_i$ in $\mathcal{G}$, the assumption here is that the central node is also the root cause. Recall one way to compute network centrality on the internet is PageRank, whereby the web is the directed graph $\mathcal{G}$, the websites are the vertices, and the links are the edges. However whereas on the web the *directed edges* are given, in this case no such assumption can be made. Thus Greenwald et al. places directed edges by sampling different vertices $\mathbf{x}_i$ in the undirected graph for interventions via $\text{do}(\mathbf{x}_i = x)$.
- **(Assumptions).** The authors make the three standard assumptions of causal identification:
 1. *(Causal assumptions):*
 - *Causal sufficiency:* for every pair of observed variables, their common cause is also observed.
 - *Causal Markov condition:* every vertex in causal graph $\mathcal{G}$ is conditionally independent of its non-descendants, given its parent.
 - *Causal faithfulness condition:* the set of independence relations derived from causal Markov condition is the exact set of independence relations.
 2. *(Causal effect identifiability):* there is an $\epsilon > 0$ s.t. for any two factors $\mathbf{x}_i$ and $\mathbf{x}_j$ in the graph $\mathcal{G}$, we have:

$$\left| p(\mathbf{x}_j = x \mid \text{do}(\mathbf{x}_i = x')) - p(\mathbf{x}_j = x) \right| > \epsilon$$



■ **Figure 27** The methods in [K19] assume the underlying causal graph $\mathcal{G}^*$ is a directed acyclic tree. Thus, causal factor identification over $\mathcal{G}$ is tree search that can be completed in $\log_2 |\mathcal{G}|$ number of interventions, where $|\mathcal{G}|$ is the number of vertices in the graph.

3. (*Reward identifiability*): for some constant $\delta > 0$, we have for every $\mathbf{x} \in \mathcal{G}$:

$$\left| \mathbb{E}[\mathcal{R}(\mathbf{y}) \mid \text{do}()] - \mathbb{E}[\mathcal{R}(\mathbf{y}) \mid \text{do}(\mathbf{x} = \mathbf{x})] \right| \geq \delta$$

4. (*Causal tree*): this method further assumes the underlying causal graph $\mathcal{G}^*$ is a directed acyclic tree as shown in fig (27). Under this graph topology, the chain and fork patterns of fig (15) are allowed, while the collider pattern is not. Under this assumption, the interventions are done in a "binary tree search" manner, thereby ensuring they can find the central node in $\log(n)$ time over vertices $\mathbf{x}_1, \dots, \mathbf{x}_n$ in the graph $\mathcal{G}$.

■ (**Definitions**).

► **Definition 61. (Directed Tree and Causal Tree)**[K19] A directed tree is a DAG whose underlying undirected graph is a tree and all its edges point away from the root. A causal tree is a causal model whose underlying causal graph is a directed tree.

► **Definition 62. (Central Node)**[K19] A central node $\mathbf{x}_R$ of an undirected tree $\mathcal{G}$ w.r.t. a distribution $\mathbf{p}(\mathbf{x}_1, \dots, \mathbf{x}_n)$ over the vertices $\mathbf{x}_1, \dots, \mathbf{x}_n$ is one for which:

$$\max_{\mathbf{x}_j \in \mathcal{N}_{\mathcal{G}}(\mathbf{x}_R)} \mathbf{p}(\mathcal{B}_{\mathcal{G}}[\mathbf{x}_R : \mathbf{x}_j]) \leq \frac{1}{2}.$$

Where $\mathcal{N}_{\mathcal{G}}(\mathbf{x}_R)$ is the neighboring vertices of $\mathbf{x}_R$ on graph $\mathcal{G}$, and $\mathcal{B}_{\mathcal{G}}[\mathbf{x}_R : \mathbf{x}_j]$ is the branch of graph $\mathcal{G}$ that can be reached from $\mathbf{x}_R$ to $\mathbf{x}_j$, once the branch from $\mathbf{x}_R$ to $\mathbf{x}_j$ has been cut. This is a definition of network centrality, that is $\mathbf{x}_R$ is "central" if the likelihood of reaching any vertex in $\mathcal{G}$ from $\mathbf{x}_j$ is less than $1/2$ for the next most central vertex.¹⁶ At least one such $\mathbf{x}_R$ is guaranteed to exist for any distribution. See example 64 below.

► **Definition 63. (Essential Graph)**[K19] The class of causal DAGs that encode the same set of conditional independence is called **The Markov Equivalence Class**. Denote the Markov equivalence class of a DAG $\mathcal{G}$ by $[\mathcal{G}]$. The essential graph of $\mathcal{G}$ has the same skeleton as $\mathcal{G}$, with directed edges $\mathbf{x}_i \rightarrow \mathbf{x}_j$ if such edge direction between $\mathbf{x}_i$ and $\mathbf{x}_j$ holds for all DAGs in $[\mathcal{G}]$, and undirected otherwise. A DAG whose essential graph has a single chain component is called a moral DAG.

■ (**Bayesian Tree Search on $\mathcal{G}$ via $\text{do}(\mathbf{x})$**). [K19] proceeds as follows:

1. Given a set of factors $\mathbf{x}_1, \dots, \mathbf{x}_i, \dots, \mathbf{x}_n$ with $\mathbf{x} \in \mathbb{R}$, and sufficient data to determine the joint distribution $\mathbf{p}(\mathbf{x}_1, \dots, \mathbf{x}_n)$. The algorithm initiates a tree $\mathcal{G}_o$ over $\mathbf{x}_1, \dots, \mathbf{x}_n$ with undirected edges, so that $\mathbf{p}(\mathbf{x}_R \text{ is } \mathbf{x}_i) = \frac{1}{n}$ for each $\mathbf{x}_i$.

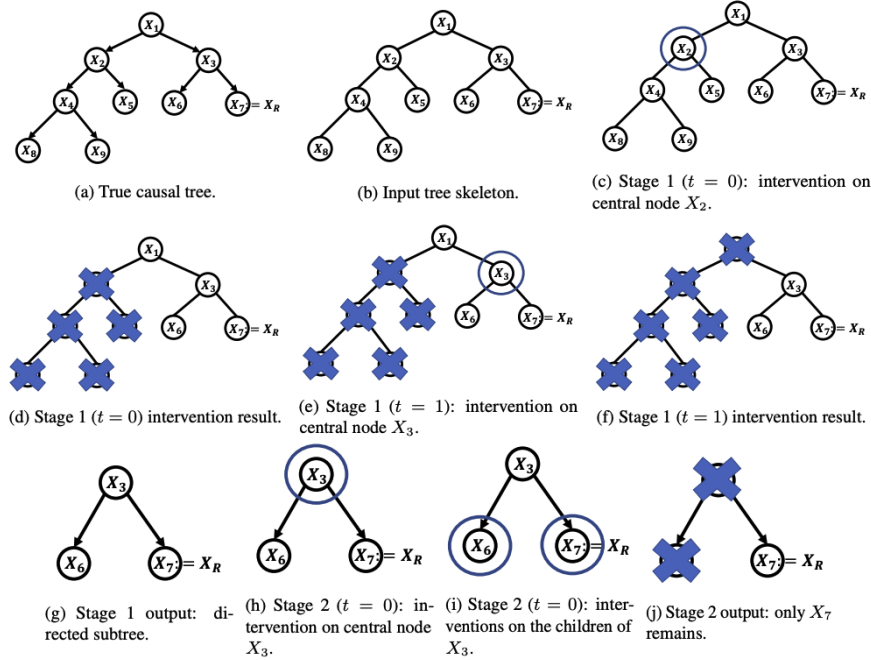
¹⁶ Recall PageRank is another way to measure network centrality.

2. They assume a prior distribution over the set of all possible graphs given this undirected tree. Then they identify the root cause, or $\mathbf{x}_R$ by exploring the *neighbors* and *branches* of each intervened vertex in $\mathcal{G}$. Some definitions follows.

- $N_{\mathcal{G}}(\mathbf{x})$: the *neighbor* of $\mathbf{x}$, i.e. in fig (27), we have $N_{\mathcal{G}}(\mathbf{x}_2) = \{\mathbf{x}_1, \mathbf{x}_4, \mathbf{x}_5\}$.
- $B_{\mathcal{G}}[\mathbf{x}_i : \mathbf{x}_k]$: a *branch* of the graph that can be reached from $\mathbf{x}_k$, when the edge from $\mathbf{x}_i$ to $\mathbf{x}_k$ has been cut.

► **Example 64. (Branch)** For example in fig (27) the branches of $\mathbf{x}_2$ are:

- * $B_{\mathcal{G}}[\mathbf{x}_2 : \mathbf{x}_1] = \{\mathbf{x}_1, \mathbf{x}_3\}$, so that the subgraph on $(\mathbf{x}_1, \mathbf{x}_3)$ is isolated from the main graph.
- * $B_{\mathcal{G}}[\mathbf{x}_2 : \mathbf{x}_4] = \{\mathbf{x}_4\}$, so the vertex $\mathbf{x}_4$ is an island.
- * $B_{\mathcal{G}}[\mathbf{x}_2, \mathbf{x}_5] = \{\mathbf{x}_5, \mathbf{x}_6\}$, so that the subgraph over $(\mathbf{x}_5, \mathbf{x}_6)$ is an island.



■ **Figure 28** Assuming there is some true underlying causal graph $\mathcal{G}^*$ in (a), the central node algorithm initializes a tree skeleton $\mathcal{G}(t=0)$ of undirected edges in (b). Then in (c) it selects $\mathbf{x}_2$ for intervention. It then searches the *isolated neighborhood* around $\mathbf{x}_2$ and compute which neighbor is most likely to contain the root cause $\mathbf{x}_R$. This likelihood is computed by eqn (35). In (d), the rest of the tree $\mathcal{G}(t=0)$ that does not likely contain $\mathbf{x}_R$ is pruned, to produce the reduced causal graph $\mathcal{G}(t=1)$. Then in (e)-(i) the process repeats on $\mathcal{G}(t=1)$. Finally a subtree containing $\mathbf{x}_R$ with confidence $1 - \delta$ is found in (j). [Lu21].

3. The algorithm initializes by selecting a random vertex $\mathbf{x}_i$ for intervention. Referencing fig (28), we assume $\mathbf{x}_i = \mathbf{x}_2$. It intervenes by setting $\text{do}(\mathbf{x}_2 = \mathbf{x}_2^{t=0})$. Once the intervention on $\mathbf{x}_2$ has occurred at round $t = 0$, we observe a sample:

$$\{\mathbf{x}_1, \mathbf{x}_3, \dots, \mathbf{x}_n\} \sim \mathbf{p}(\mathbf{x}_1, \mathbf{x}_3, \dots, \mathbf{x}_n \mid \text{do}(\mathbf{x}_2 = \mathbf{x}_2^{t=0})).$$

Now it computes the likelihood that the root cause $\mathbf{x}_R$ is in *any* isolated neighborhood around $\mathbf{x}_2$. That is for every vertex $\mathbf{x}_k$ in the neighborhood $N_{\mathcal{G}(t=0)}(\mathbf{x}_2) \cup \{\mathbf{x}_2\}$, we can construct the branch $B_{\mathcal{G}(t=0)}[\mathbf{x}_2 : \mathbf{x}_k]$ by severing the ties from $\mathbf{x}_k$ to $\mathbf{x}_2$. Then the likelihood that any $\mathbf{x}_a$ in

this branch is the root cause $\mathbf{x}_R$ is given by:

$$\forall \mathbf{x}_a \in \mathcal{B}_{\mathcal{G}}[\mathbf{x}_2 : \mathbf{x}_k],$$

$$p(\mathbf{x}_R \text{ is } \mathbf{x}_a \mid \text{do}(\mathbf{x}_2), x_1, x_3, \dots, x_n) \sim \begin{cases} p(\mathbf{x}_R \text{ is } \mathbf{x}_a) \frac{p(\mathbf{x}_k = \mathbf{x})}{p(\mathbf{x}_k = \mathbf{x} \mid \mathbf{x}_2 = \mathbf{x}_2^{t=0})} & \text{if } \mathbf{x}_k \in N_{\mathcal{G}(t=0)}(\mathbf{x}_2) \\ p(\mathbf{x}_R \text{ is } \mathbf{x}_a) & \text{if } \mathbf{x}_k = \mathbf{x}_2. \end{cases} \quad (35)$$

Here the quantity $p(\mathbf{x}_R \text{ is } \mathbf{x}_a)$ is the prior likelihood that any $\mathbf{x}_a$ is $\mathbf{x}_R$. And note that this expression translates intervention distribution query on the left hand side with $p(\cdot \mid \text{do}(\mathbf{x}_2), \dots)$, to an observation distribution with term $p(\cdot \mid \mathbf{x}_2 = \mathbf{x}_2^{t=0})$ on the right hand side.

- The process above eliminates all isolated neighborhoods around $\mathbf{x}_2$ with the exception of one - where the root cause $\mathbf{x}_R$ is most likely to located. Then it searches this likely neighborhood $\mathcal{G}(t = 1)$. The search runs until the root vertex $\mathbf{x}_R$ is determined with $1 - \delta$ confidence. The algorithm skeleton is given in 4.

■ **Algorithm 4** Central Node Algorithm, adapted from [K19]

Require: Observational tree $\mathcal{G}_o$ with $|\mathcal{G}_o| = n$, confidence parameter δ

- 1: $t \leftarrow 0$
- 2: $\mathcal{G}(0) = \mathcal{G}_o$
- 3: $p(\mathbf{x}_R \text{ is } \mathbf{x}_a)^o \leftarrow \frac{1}{n} \quad \forall \mathbf{x}_a \in \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$
- 4: **while** $\max_{a \in \{1, \dots, n\}} p(\mathbf{x}_R \text{ is } \mathbf{x}_a)^t \leq 1 - \delta$ **do**
- 5: $t \leftarrow t + 1$
- 6: Identify central vertex $\mathbf{x}_o$ of $\mathcal{G}(t - 1)$ w.r.t $p(\mathbf{x}_R \text{ is } \mathbf{x}_a)^{t-1}$
- 7: Intervene on vertex $\mathbf{x}_o$ and observe sample $\{x_1, \dots, x_n\} / \{\mathbf{x}_o\}$
- 8: Update posterior distribution according to eqn (35)
- 9: Using posterior locate neighborhood around $\mathbf{x}_t$ where $\mathbf{x}_R$ is likely to be found
- 10: $\mathcal{G}(t) \leftarrow \mathcal{B}_{\mathcal{G}(t-1)}[\mathbf{x}_o : \mathbf{x}_t]$
- 11: **end while**
- 12: **return** vertex remaining in $\mathcal{G}(t)$ as the root cause $\mathbf{x}_R$ of $\mathcal{G}$

The authors of [K19] showed that the running time of the algorithm above is $T \leq \log_2(|\mathcal{G}|)$, and its expected running time is constant factor away from the optimal running time by $\mathbf{E}[T] \leq 2\mathbf{E}[T_{opt}]$. The paper [K19] then extends the CN algorithm to arbitrary graphs with the central-node upper confidence bound algorithm (CN-UCB). The algorithm skeleton is presented in algorithm 5.

■ **Algorithm 5** Central-Node Upper Confidence Bound (CN-UCB) method [Lu21]

Require: identifiability parameter ϵ, δ, B, m

Require: Tree skeleton $\mathcal{G}_o$ with $|\mathcal{G}_o| = n$, Joint distribution $\mathbf{p}(\mathbf{x}_1, \dots, \mathbf{x}_n)$

- 1: Perform $\text{do}()$ B times, collect $\mathcal{R}_1, \dots, \mathcal{R}_B$
- 2: Obtain reward estimate for empty intervention $\text{do}()$:
- 3: $\tilde{\mathcal{R}} \leftarrow \frac{1}{B} \sum_{b=1}^B \mathcal{R}_b$
- 4: **Stage 1:** find a directed subtree that contains $\mathbf{x}_o$:
- 5: $\tilde{\mathcal{G}}_o \leftarrow \text{FindSubTree}(\mathcal{G}_o, n, B, \epsilon, \delta, \tilde{\mathcal{R}})$
- 6: **Stage 2:** find key factor $\mathbf{x}_R$ that generates the reward:
- 7: $\mathbf{x}_R \leftarrow \text{FindKeyFactor}(\tilde{\mathcal{G}}_o, n, B, \delta, \tilde{\mathcal{R}})$
- 8: **Stage 3:** apply UCB algorithm on:

$$\{\text{do}(\mathbf{x}_R = \mathbf{x}_i) \mid i = 1, \dots, n\}$$

for m rounds.



References

- 1 *Covenant*. Cornell Law, 2025.
- 2 *Covenant that runs with the land*. Cornell Law, 2025.
- 3 *Exploring the three rules of do-calculus in plain language and deriving the backdoor adjustment formula by hand*. andrewheiss, 2025.
- 4 *The Three Layer Causal Hierarchy*. UCLA, 2025.
- 5 Lee S. Bareinboim E., Zhang J. *An Introduction to Causal Reinforcement Learning*. 2025.
- 6 Lu C. *Learning Causal Representations for Generalization and Adaptation in Supervised, Imitation, and Reinforcement Learning*. 2022.
- 7 B Schölkopf J Peters, D Janzing. *Elements of Causal Inference*. The MIT Press, 2017.
- 8 Greenwald K. Sample efficient active learning of causal trees. *NeurIPS*, 2019.
- 9 X. Ling. *Metheus: Fair Accountable Transparent Machine Intelligence Audit*. 2025.
- 10 Y Lu. Causal bandits with unknown graph structure. *NeurIPS*, 2021.
- 11 A. Zaldivar P. Barnes L. Vasserman B. Hutchinson E. Spitzer I. D. Raji T. Gebru M. Mitchell, S. Wu. Model cards for model reporting. *FAT*, (220-229), 2019. doi:10.1145/3287560.3287596.
- 12 Oddur Kjartansson Mahima Pushkarna, Andrew Zaldivar. Data cards: Purposeful and transparent dataset documentation for responsible ai. *FAccT*, 2022. doi:10.1145/3531146.3533231.
- 13 J. Pearl. *Causality: Models, Reasoning, and Inference*. 2009.
- 14 J Pearl. The foundation of causal inference. *ASA*, 2010.
- 15 J. Pearl. The do-calculus revisited. *UAI*, 2012.
- 16 Benjamin C. Pierce. *Types and Programming Languages*. The MIT Press, 2002.
- 17 Benjamin C. Pierce. *Software Foundations, Volume 2: Programming Language Foundations*. The MIT Press, 2017.
- 18 Bedi S. Fidelity of medical reasoning in large language models. *Research Letter Health Informatics*, 8(8), 2025. doi:10.1001/jamanetworkopen.2025.26021.
- 19 Richard F. Schmidt. *Software Engineering Architecture-driven Software Development*. Elsevier Inc, 2013.
- 20 Hassabis D. Silver D., Huang A. Mastering the game of go with deep neural networks and tree search. *Nature*, 2016.
- 21 Barton AG Sutton RS. *Reinforcement Learning: An Introduction*. The MIT Press, 2015.
- 22 X. Ling W. Powell. *FairCovenant: Smart Trade Hubs to Accelerate Fair Global Trade*. 2025.
- 23 Zhu ZM. Offline reinforcement learning with causal structured world models. *Lambda*, 2022.